CycloneDX

Authoritative

Guide to AI/ML-BOM

Drive Transparency, Compliance, and Security Across the AI Supply Chain

OWASP

First Edition

Table of Contents

About the Guide

CycloneDX is a modern standard for the software supply chain. It has been ratified as ECMA-424 by Ecma International.

The content in this guide results from the work of the CycloneDX AI/ML Working Group with continuous community feedback and input from leading experts in the field. This guide would not be possible without valuable feedback from peers at the CycloneDX Industry Working Group (IWG), the CycloneDX Core Working Group (CWG), the many CycloneDX Feature Working Groups (FWG), Ecma International Technical Committee 54, and a global network of contributors and supporters.

Copyright and License

First Edition (Revision 1), 10 June 2026

Version	Changes	Updated On	Updated By
First Edition, r1	Updates to appendices and examples	2026-06-10	CycloneDX AI/ML Working Group
First Edition	Initial Release	2026-03-03	CycloneDX AI/ML Working Group

Preface

Welcome to the Authoritative Guide series by the OWASP Foundation and OWASP CycloneDX. In this series, we aim to provide comprehensive insights and practical guidance, ensuring that security professionals, developers, and organizations alike have access to the latest best practices and methodologies.

At the heart of the OWASP Foundation lies a commitment to inclusivity and openness. We firmly believe that everyone deserves a seat at the table when shaping the future of cybersecurity standards. Our collaborative model fosters an environment where diverse perspectives converge to drive innovation and excellence.

In line with this ethos, the OWASP Foundation has partnered with Ecma International to create an inclusive, community-driven ecosystem for the development of security standards. This collaboration empowers individuals to contribute their expertise and insights, ensuring that standards like CycloneDX reflect the collective wisdom of the global cybersecurity community.

One standout example of this model is OWASP CycloneDX, which has been ratified as an Ecma International standard and is now known as ECMA-424. By leveraging the strengths of both organizations, CycloneDX serves as a cornerstone of security best practices, providing organizations with a universal standard for software and system transparency.

As you embark on your journey through this Authoritative Guide, we encourage you to engage actively with the content and join us in shaping the future of cybersecurity standards. Together, we can build a safer and more resilient digital world for all.

Andrew van der Stock
Executive Director, OWASP Foundation

The Innovative History of OWASP CycloneDX

OWASP CycloneDX has carved a legacy steeped in innovation, collaboration, and a commitment to openness. OWASP continues to advance software and system transparency standards, prioritizing capabilities that facilitate risk reduction.

October 2025

OWASP CycloneDX v1.7

First specification supporting citations that improve traceability, attribution, and auditability, and comprehensive support for patents and patent families to address intellectual property transparency.

June 2024

International Standardization

CycloneDX v1.6 ratified as an Ecma International standard and published as ECMA-424.

April 2024

OWASP CycloneDX v1.6

First specification to support cryptographic assets for Post-Quantum Cryptography (PQC) readiness and first general-purpose attestation specification to digitally transform audit and attestation workflows.

December 2023

Ecma TC54 Established

First working group chartered with holistic supply chain goals of standardizing core data formats, APIs, and algorithms that advance software and system transparency.

June 2023

OWASP CycloneDX v1.5

First specification to support AI Transparency, configuration and data components, and formulation describing how components were created, tested, trained, evaluated, and deployed.

January 2022

OWASP CycloneDX v1.4

First specification to introduce vulnerability sharing and transparency, including Vulnerability Disclosure Reports (VDR) and Vulnerability Exploitability eXchange (VEX).

May 2021

OWASP CycloneDX v1.3

First specification to incorporate support for composition completeness surpassing NTIA's framing of "known unknowns".

May 2020

OWASP CycloneDX v1.2

First specification to incorporate SWID (ISO/IEC 19770-2:2015) and services into inventory including data classifications, providers, and relationships between services and components.

March 2019

OWASP CycloneDX v1.1

First specification with complete pedigree support describing component lineage and the commits, patches, and diffs which make a forked version unique.

March 2018

OWASP CycloneDX v1.0

First general-purpose, security-focused Bill of Materials standard supporting software and hardware components. Introduced the world to Package-URL for software security use cases.

Source: https://tc54.org/history

Acknowledgements

The CycloneDX Authoritative Guide to ML-BOM represents the collective effort and expertise of many individuals and organizations committed to advancing transparency and security in artificial intelligence and machine learning systems.

Primary Contributors

This guide was developed by the **CycloneDX AI/ML Working Group**, whose members dedicated countless hours to research, discussion, writing, reviewing and refining the content to ensure it meets the highest standards of technical accuracy and practical applicability.

We would like to specifically acknowledge the following individuals for their much appreciated contributions:

- Matt Rutkowski, IBM, CycloneDX Project Maintainer, Chair AI/ML Work Group
- Steve Springett, Chair of CycloneDX Standard, Founder of Ecma TC54, Chair of OWASP Global Board of Directors
- Jan Kowalleck, CycloneDX Project Co-Lead
- Michael Boone, NVIDIA
- Jessica Butler, NVIDIA
- Pratyusha Maiti, NVIDIA
- Saquib Saifee, IBM
- Colin Gigool, IBM
- Aliza Heching, IBM
- Nagalakshmi Satyanarayanan, IBM
- Pavel Shukhman, Reliza

Community Support

We extend our gratitude to the broader CycloneDX community for their invaluable feedback and contributions:

- **CycloneDX Core Working Group (CWG)** - for technical guidance and specification alignment
- **Ecma International Technical Committee 54 (TC54)** - for standardization support and governance

Open Source Spirit

This guide embodies the open-source ethos of the OWASP Foundation and CycloneDX project. We thank all contributors, reviewers, and users who continue to improve and evolve this resource through their feedback and participation.

The development of this guide would not have been possible without the collaborative spirit and dedication of the global CycloneDX community.

Introduction

CycloneDX is a modern standard for the software supply chain. At its core, CycloneDX is a general-purpose Bill-of-Materials (BOM) standard capable of representing software, hardware, services, and other types of inventory.

CycloneDX is notably an OWASP flagship project, has a formal standardization process and governance model through Ecma Technical Committee 54, and is supported by the global information security community.

What is an ML-BOM?

An ML-BOM (Machine Learning Bill of Materials) is a CycloneDX BOM document designed to address the unique complexities and risks of AI/ML systems. It provides a detailed inventory of all components, configurations, and processes involved in the development, training, deployment, and hosting (i.e., via hardware/software stacks and frameworks) of a machine learning model.

The primary purpose of an ML-BOM is to ensure transparency, traceability, security, and compliance throughout an ML model's lifecycle.

Why ML-BOMs are Important

ML-BOMs address critical challenges in the machine learning supply chain:

- **Security & Vulnerability Management**: Help identify security risks, such as malicious (open-source) models or vulnerable dependencies, before they are integrated into production applications.
- **Governance & Compliance**: Provide documentation for audits or formal informational requests based upon requirements from emerging global AI regulations such as the European Union's Cyber Resilience Act (EU CRA), including specifics for AI models and systems from the complementary EU AI Act, as well as for voluntary, guidance-focused frameworks such as the NIST AI Risk Management Framework.
- **Risk Mitigation**: Enable teams to track data lineage, helping to identify and eliminate potential data quality issues, privacy risks, or unwanted biases that could affect the model's performance and fairness.
- **Reproducibility & Explainability**: Show adherence to software development lifecycle best practices by providing a detailed record of components and training processes such that developers are able to reproduce models (via training from datasets) and their benchmarks in order to validate claims of model accuracy and adherence to ethical considerations.

Core Concepts and Considerations

Key Components of an ML-BOM

An ML-BOM typically documents the identifying elements, architecture, components, and its supply chain along with any configurations and developmental or executional considerations, inclusive of the following areas:

- **Model identifiers**: Identifying information such as the model's Package URL (PURL) (e.g., from Huggingface pkg:huggingface/distilbert-base-uncased@043235d6088ecd3dd5fb5ca3592b6913fd51602) or other domain-specific identifiers within other registries.
- **Model metadata**: Descriptive details such as the model's name, version, license, developer, purpose, use cases, architecture, (hyper)parameters and any additional identifying elements.
- **Model architecture**: Description of the composition of the model's neural network including configurations, layers, input/output parameters, attention mechanisms, etc. used at network processing stages.
- **Datasets**: Description of datasets, as CycloneDX data components, used for training and testing of the associated model. This includes data sources, selection criteria, acquisition methods, preprocessing steps, and more.
- **Tokenizers and prompt templates**: Descriptive details of specific tokenizers (e.g., libraries, files, configurations) and prompt templates used to train and/or interact with the model during runtime.
- **Hardware, software & frameworks**: A list of all hardware and software components including libraries, packages, frameworks (e.g., TensorFlow, PyTorch, Huggingface), along with specific versions and associated licenses used in aspects of the model's lifecycle. This informational category may also include operational and application aspects of models (perhaps as agents) used within compositional frameworks and workflows, along with the protocols used for communication.
- **Training & testing details**: Information about the computational environment and systems (software, hardware, operating system, and GPUs) used for training or evaluation along with necessary configurations, hyperparameters, and evaluation metrics.
- **Intended use & ethical considerations**: Documentation of the model's intended use, known limitations, safety guardrails, and ethical considerations.
- **Environmental impacts**: Documentation of the resource needed to train or execute the model which have an environmental impact or cost (e.g., data center energy and water cooling cost details).

ML-BOM Design and Best Practices

Overview

A Machine Learning Bill-of-Materials (MLBOM or ML-BOM) is an object model to describe a machine learning model, its compositional assets, and other descriptive information often used to assess risk and compliance. Support for MLBOM is included in CycloneDX v1.5 and higher.

An MLBOM relies on many of the common, core elements of the CycloneDX schema, as well as unique aspects specific to ML components, their architectures, metadata, training, and other information used to gauge regulatory compliance.

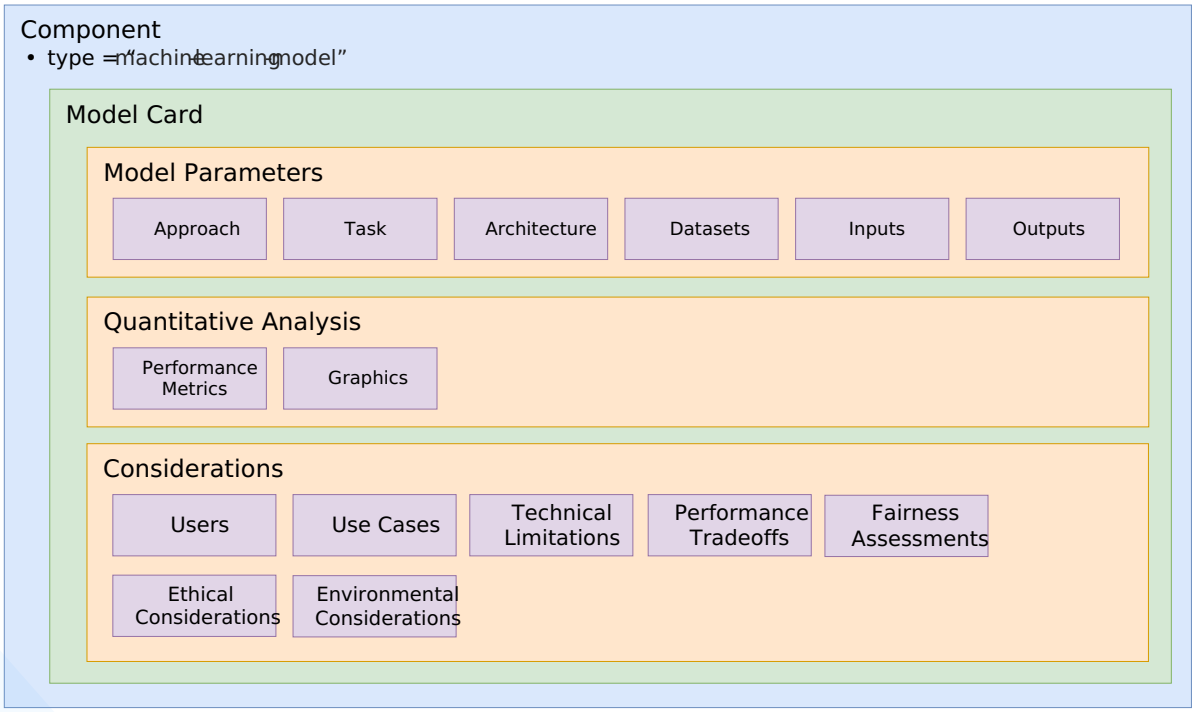
This guide will provide specifics and best practices for conveying ML-related information using the CycloneDX schema.

The Core Concepts listed in the previous section will be used to provide details, best practices, and examples for providing the corresponding information using CycloneDX schema objects.

For convenience, here are links to the specific sections for each of those informational areas:

Anatomy of an ML-BOM

In CycloneDX, a model is considered a component where general best practices for providing information such as component identification, metadata, provenance, pedigree, etc. should be followed as documented in the CycloneDX Authoritative Guide to SBOM.



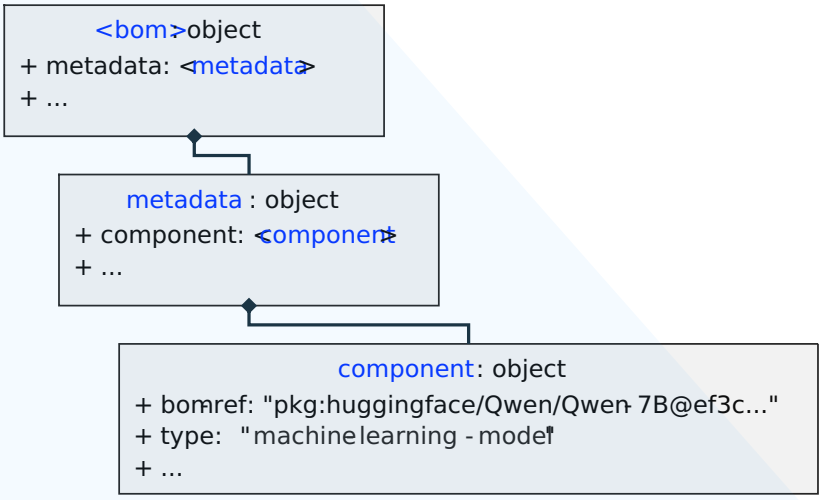
4

Declaring ML models

Describing models as components

A model should always be declared as a CycloneDX component. If the model itself is the subject of the BOM, then the BOM is considered an ML-BOM, and the component representing it would be declared in the top-level BOM metadata object.

The object model's pseudo-schema would look something like this:



> ***Note*** *In all example JSON pseudo-code, lines with: "// ..." indicate that additional fields or elements may be added at this position. This is a documentation placeholder and is not valid JSON; it would not appear in actual implementations.*

Example: Declaring an ML model in an ML-BOM

The CycloneDX JSON pseudocode below shows how an ML model would be declared as the "subject" component of an ML-BOM within the top-level metadata:

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:ec45525e-516c-4405-9de3-4fbdaef7f09a",
  "version": 1,
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      "purl": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9c57b252f3149c1408daf4d649ec8b6c85",
      "version": "ef3c5c9c57b252f3149c1408daf4d649ec8b6c85",
      "licenses": [
        {
          "license": {
            "name": "Tongyi Qianwen LICENSE AGREEMENT",
            "text": {
              "content":
"By clicking to agree or by using or distributing any portion or element of the Tongyi Qianwen
Materials, ..."
            }
          }
        }
      ]
      // ...
    },
    // ...
  }
  // ...
}
```

Field discussion

- **bom-ref** - Please note the bom-ref value includes the first seven characters of the larger hash value from the purl component identifier which is sufficient for local identification within the BOM itself.
- **license** - The licenses object shown in the example is a "custom" license which, in this case, we chose to provide the unencoded license text. It is preferable, when possible to use an SPDX license identifier and supply it in the id field of the license (e.g., "license": { "id": "Apache-2.0" }).

Model repositories as components

When referencing an ML model as a component, it typically means you are referencing a **model repository** comprised of metadata and a set of files (e.g., pre-trained tensor data in various formats, model configurations, tokenizers, tokenizer configurations, prompt templates, Python code, etc.) which would be selectively used with various, compatible AI or ML applications and frameworks.

If possible, these model repositories should be treated as a software "package" in a Software Bill of Materials (SBOM) when declaring them as machine-learning-model type CycloneDX components.

Example: CycloneDX for the Qwen-7B model repository

The following example shows how the Hugging Face Qwen/Qwen-7B model repository would be declared as a CycloneDX component of type machine-learning-model in a CycloneDX ML-BOM as its subject component.

Since the model repository is hosted on Hugging Face, the Huggingface package type may be used Package URL specification to identify the model.

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      "purl": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9c57b252f3149c1408daf4d649ec8b6c85",
      "group": "Qwen",
      "manufacturer": {
        "name": "Alibaba Cloud",
        "url": [ "https://www.alibabacloud.com" ]
      },
      "supplier": {
        "name": "Hugging Face",
        "url": [ "https://huggingface.co" ]
      },
      "name": "Qwen/Qwen-7B",
      "version": "ef3c5c9c57b252f3149c1408daf4d649ec8b6c85",
      "description": "Qwen-7B is a Transformer-based large language model, which is pretrained on a
large volume of data, including web texts, books, codes, etc.",
      "externalReferences": [
        {
          "type": "vcs",
          "url": "https://huggingface.co/"
        },
        {
          "type": "model-card",
          "url": "https://huggingface.co/Qwen/Qwen-7B"
        }
      ],
      "modelCard": {
        // ...
      }
    }
  }
}
```

Field discussion

This section provides best practice guidance on how the component fields were filled out for this example.

- **bom-ref** - Since a PURL is available, it can also be used as the bom-ref.
- **purl** - The Package URL (PURL) follows the Huggingface package type using a commit hash.
- **manufacturer** - The name of the company which built the Qwen model.
- **group** - In this example, we chose to include the optional group field to acknowledge the specific model repository is part of the Qwen family of models.
- **name** - The model name reflects how the model is identified under Hugging Face using the <owner_name>/<repository_name> format.

- **version** - Models are not always versioned in the way software packages are (e.g., using semver format); however, within repositories such as Huggingface, the version is determined by its version control system's *commit hash*, *tag*, or *branch*. In the above example, the model's commit hash matches the purl value.
- **externalReferences** - Used to provide unambiguous links to component's model repository and originating model card.
    - **vcs** - Provides a link to the version control system (i.e., the model provider aka. supplier). In this example, Hugging Face is used to affirm the associated PURL identifier.
    - **model-card** - Provides a link to the model's Hugging Face model card which is comprised of mostly unstructured information in the form of a markdown file (i.e., README.md).
      *The CycloneDX representation of model card information will be detailed in a subsequent section.

Model identifier(s)

As you can see in the above example, the component has a bom-ref that is also a valid Package URL (PURL) for a "Qwen-7B" model hosted in a Huggingface model repository using the Hugging Face PURL type. When a valid purl value is available for a model, it is recommended that it also be used as its component's bom-ref.

If the model being described by an ML-BOM is instead hosted in a GitHub repository, it can also be referenced using a GitHub Package URL. For example, the ONNX vision model: tiny-yolov2 would have a github PURL type.

Example: JSON for model component with GitHub PURL

Note: The derivative bom-ref, based upon the PURL, is also shown.

```
"component":
{
  "type": "machine-learning-model",
  "purl": "pkg:github/onnx/models@4c46cd00fbdb7cd30b6c1c17ab54f2e1f4f7b177#validated/vision/
object_detection_segmentation/tiny-yolov2/model",
  "bom-ref": "pkg:github/onnx/models@244fd47#tiny-yolov2/model",
  // ...
}
```

Adding domain-specific identifiers

Organizations that produce BOMs for hardware or software components they produce may have multiple domain-specific identifiers for the same component. In these cases, it is best practice to register (reserve) an official namespace for these domains with the CycloneDX Property Taxonomy, which is the authoritative source of official namespaces used in CycloneDX properties.

Example: domain-specific identifiers

The following example shows how a registered name for a fictional company, ACME, which registered the namespace acme, could provide a property to identify one of its internal ML models.

```
"component": {
  "properties": [
    {
      "name": "acme:research:model:llm:id",
      "value": "MODEL-ID-12345-INTERNAL"
    },
    // ...
  ],
  // ...
}
```

Identifying a specific model quantization

Some model repositories may contain different quantizations to choose from, which optimize for different target inference runtimes and hardware footprints.

In general, these are referenceable as (often single) files within a model repository, each of which can be described as a CycloneDX component, as shown in the next section Describing a model repository as a CycloneDX assembly.

Example: Qwen/Qwen3-8B-GGUF

This example uses the model repository Qwen/Qwen3-8B-GGUF, which contains several quantizations of the Qwen3-8B model (published originally in a non-quantized format elsewhere) in GGUF format.

These quantized GGUF models are each individual files in the repository:

- Qwen3-8B-Q4_K_M.gguf
- Qwen3-8B-Q5_0.gguf
- Qwen3-8B-Q6_K.gguf
- Qwen3-8B-Q8_0.gguf

Each can be specifically identified in a CycloneDX component using a Package URL (PURL). For example, the Qwen3-8B-Q4_K_M.gguf model would be declared as follows:

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:1ad676cb-6b40-4068-ae91-ebd1533dbf58",
  "version": 1,
  // ...,
  "components": [
    {
      "name": "Qwen3-8B-Q4_K_M.gguf",
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen3-8B-GGUF@7c41481#Qwen3-8B-Q4_K_M.gguf",
      "purl": "pkg:huggingface/Qwen/Qwen3-8B-GGUF@7c41481f57cb95916b40956ab2f0b139b296d974#Qwen3-8B-
Q4_K_M.gguf",
      "version": "7c41481f57cb95916b40956ab2f0b139b296d974",
      // ...
    }
  ],
  // ...
}
```

Providing model release notes

It is important to disclose information regarding a model's release. This is accomplished by utilizing the CycloneDX component's releaseNotes object and its fields.

Example: release notes

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...
  "metadata": {
    "component": {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      // ...
      "releaseNotes": {
          "type": "major",
          "title": "Qwen 7B initial release",
          "timestamp": "2023-08-03T15:30:00Z",
          "notes": [
            {
              "locale": "en-US",
              "text": "United States (US), English release date."
            },
            // ...
          ]
        }
      ]
    },
    // ...
  }
}
```

Field discussion

- **type** - the type has the value machine-learning-model since the single file contains all the information (e.g., default configuration parameters, references to architectures and tokenizers, prompt template, etc.) needed to run the model in GGUF inference frameworks.

Describing a model repository as a CycloneDX assembly

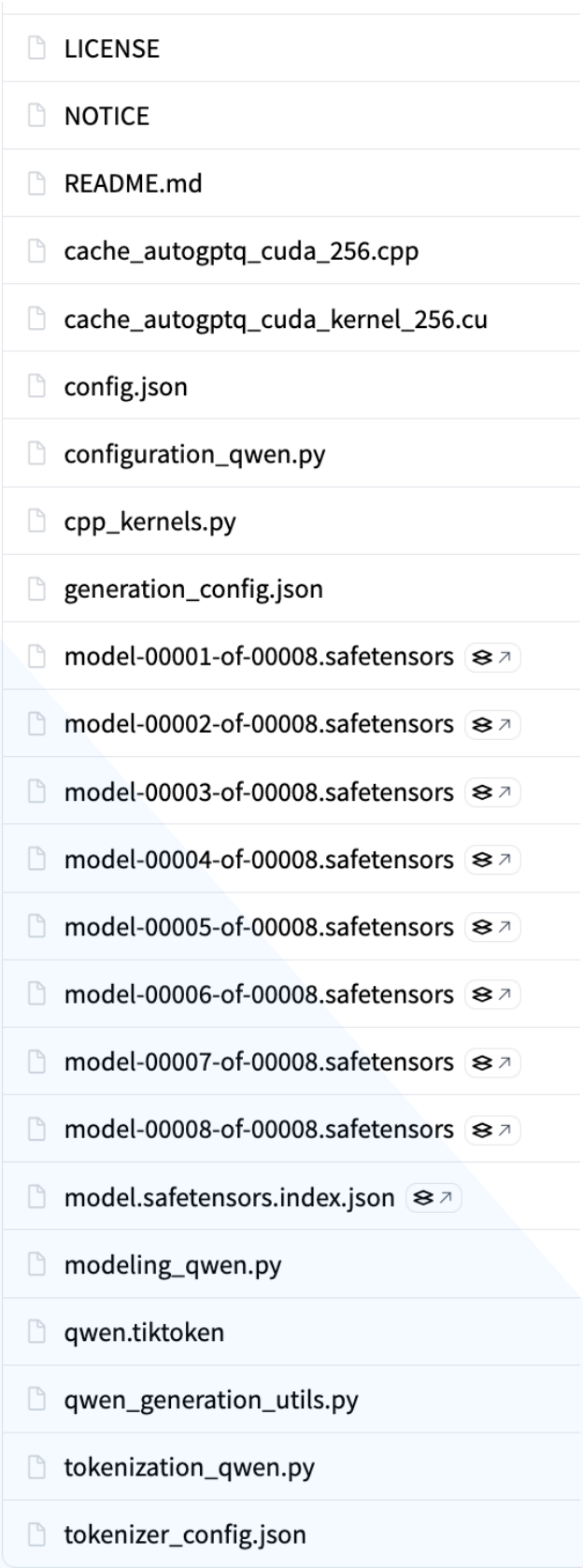
CycloneDX allows for declarations of software compositions (e.g., hardware products, software applications, packages, libraries, archives, etc.).

In the case of a model repository like those hosted in Hugging Face, one can describe the files that comprise it as a composition with an ML-BOM. Specifically, it would be declared as a composition of an assembly type.

Specifically, a component entry would be created for each file and declared in the ML-BOM's components array hierarchically under the model's component then declare the assembly relationship within within the BOM's compositions array under assemblies by providing the bom-ref link to the model component that contains the hierarchy of the constituting (file) components within the model repository.

Example: Qwen/Qwen-7B model repository files

If we look inside the repository for the Qwen/Qwen-7B model in Huggingface, we see the complete list of files that make up the "model" in its repository:



CycloneDX for the Qwen/Qwen-7B assembly

The simplified JSON below shows how to declare a few files from the model repository's complete file list within the BOM's component under the model's metadata declaration.

> ***Note**: In the JSON below, we use the Package URL (PURL) syntax to provide the additional path (with the model repository or "package") to each individual file by appending it using the # hash symbol as a separator. Also, notice that the commit hash (identifier) varies per file.*

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
    // ...,
    "metadata":
    {
      "component":
      {
        "type": "machine-learning-model",
        "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
        // ...
        "components": [
          {
              "type": "file",
              "name": "config.json",
              "description": "Model configuration file using the 'QWenLMHeadModel' model class in
Hugging Face Transformers",
              "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@e7a368b#config.json",
              "purl": "pkg:huggingface/Qwen/
Qwen-7B@e7a368b0774370edec29674e7c51f52fc7663f59#config.json",
              // ...
          },
          {
              "type": "file",
              "name": "configuration_qwen.py",
              "description": "Python 'QWenConfig' class implementation for the Qwen-7B model using
Hugging Face Transformers",
              "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@a6ca629#configuration_qwen.py",
              "purl": "pkg:huggingface/Qwen/
Qwen-7B@a6ca629d063f56f34d184852301e8852a7afbd58#configuration_qwen.py",
              // ...
          },
          {
              "type": "data",
              "name": "model-00001-of-00008.safetensors",
              "description": "Model tensor data (01 of 08)",
              "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@abcb6d6#model-00001-of-00008.safetensors",
              "purl": "pkg:huggingface/Qwen/
Qwen-7B@abcb6d6d8ec63ce606f816e2d08072da6309f965#model-00001-of-00008.safetensors",
              "data": [
                {
                  "type": "dataset",
                  "contents": {
                    "url": "https://huggingface.co/Qwen/Qwen-7B/blob/main/model-00001-
of-00008.safetensors"
                  }
                }
              ]
              // ...
          },
          {
              "type": "data",
              "name": "model-00002-of-00008.safetensors",
              "description": "Model tensor data (02 of 08)",
```

```
            "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@abcb6#model-00002-of-00008.safetensors",
            "purl": "pkg:huggingface/Qwen/
Qwen-7B@abcb6d6d8ec63ce606f816e2d08072da6309f965#model-00002-of-00008.safetensors",
            "data": [
              {
                "type": "dataset",
                "contents": {
                  "url": "https://huggingface.co/Qwen/Qwen-7B/blob/main/model-00002-
of-00008.safetensors"
                }
              }
            ]
            // ...
          },
          // ...
        ]
      }
    }
  // ...
}
```

Then the model component's new hierarchy of composing files would be described as an assembly composition as follows:

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "composition": [
    {
      "aggregate": "complete",
      "assemblies": [
        "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      ]
    }
  ],
  // ...
}
```

Discussion of composition fields

- **aggregate** - Note the composition aggregate value is assigned to be "complete" since all constituent files are known and declared in the ML-BOM as part of the model component's components hierarchy.

Declaring a model's pedigree

ML models are often derived from existing, pre-trained models to optimize performance, reduce resource consumption, and adapt to specialized tasks without training from scratch. Some reasons for this include:

- **Fine-Tuning**: Specialized adaptation where a general model (e.g., LLM) is retrained on a smaller, targeted dataset to improve performance for specific domains.
- **Quantization**: Reduces model size and increases inference speed by mapping parameters to lower-precision tensor formats (e.g., from FP32 to int8 or Q4_K_M precision), which also lowers energy consumption for edge devices.
- **Format Conversions**: Transforming models between frameworks (e.g., PyTorch to ONNX) ensures interoperability, allowing deployment on different frameworks and accelerators.
- **Pruning**: Derives a smaller model by removing redundant or less important parameters (weights) that do not significantly contribute to output accuracy.
- **Adapters**: Adding small, trainable layers (adapters) to a frozen base model to adapt it to new tasks without changing the original, large model weights, saving on storage for multi-task scenarios.

It is important to capture any of these transformations in the model's lineage ("pedigree") or in an ML-BOM. This should be accomplished via the CycloneDX pedigree object and describing a model's ancestors as a hierarchical graph.

Example: Declaring the finetuning of llama3 model for a coding variant

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata": {
    "component": {
      "type": "machine-learning-model",
      "name": "unsloth/Llama-3.2-3B-Instruct",
      "purl": "pkg:huggingface/unsloth/Llama-3.2-3B-Instruct@1.0.0",
      "bom-ref": "pkg:huggingface/unsloth/Llama-3.2-3B-Instruct@1.0.0",
      "publisher": "Unsloth",
      "description": "A pre-optimized, specialized versions of the meta-llama/Llama-3.2-3B-Instruct
model designed to work seamlessly with Unsloth's training framework",
      // ...,
      "pedigree": {
        "ancestors": [
          {
            "type": "machine-learning-model",
            "name": "meta-llama/Llama-3.2-3B-Instruct",
            "publisher": "Meta",
            "purl": "pkg:huggingface/meta-llama/Llama-3.2-3B-Instruct",
            "description": "The original base model from Meta Llama used for fine-tuning."
          }
        ]
      }
    }
  }
}
```

Field discussion

- **ancestors** - ancestors entries are themselves CycloneDX component objects. It should be noted that these models may have their own ML-BOMs, which can be located via their identifiers (e.g., purl) or via externalReferences for readers to follow.

Declaring known descendants

If, at the time an ML-BOM is created for a model, its downstream model variants (e.g., finetunings, quantizations, etc., derived from the model) are known, these can also be recorded within the pedigree object as descendants in a similar manner.

Model cards

A model card describes the intended uses of a machine learning model and potential limitations, including biases and ethical considerations. Model cards typically include the training parameters, the datasets used to train the model, performance metrics, and other relevant data useful for ML transparency. This object *SHOULD* be specified for any component of type machine-learning-model and must not be specified for other component types.

Throughout the model card sections of this guide, we will show how to use the existing schema to encode information from model cards in a more current and robust way.

Overview

This section describes the design and best practices when providing information for a CycloneDX modelCard in an ML-BOM as part of the model's CycloneDX component definition.

For convenience, here are links to the specific sections for each of those informational areas:

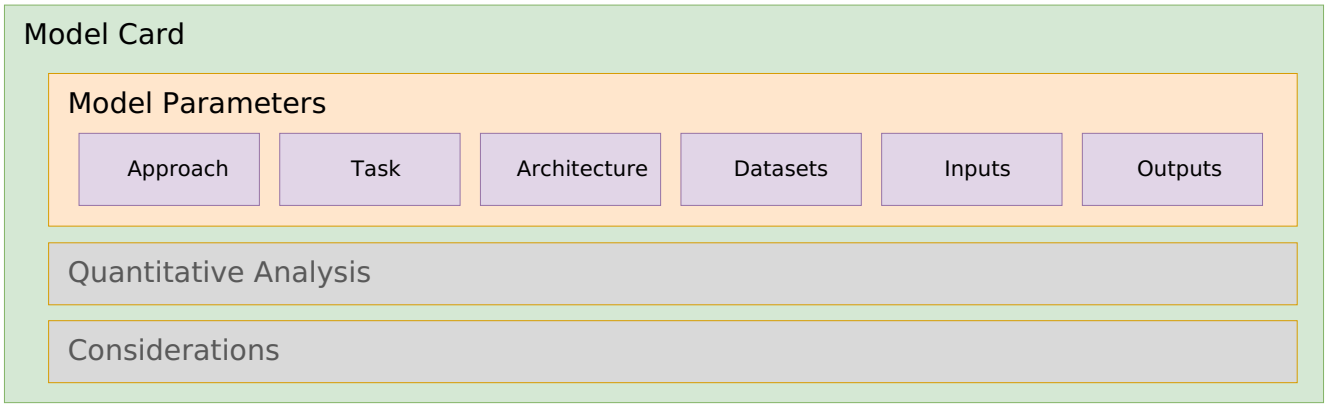
- Including manufacturing information for the ML model

Design notes

Please note that at the time of initial development, the CycloneDX model card schema was heavily influenced by Tensorflow ModelCard Toolkit and specifically its ModelCard fields since it was one of the few frameworks available for reference.

Since that time, the AI/ML landscape has progressed rapidly, both in the design of model architectures and in the increased emphasis on providing model disclosure to comply with governmental regulations. *In order to account for these changes, the CycloneDX community plans to provide significant improvements and normative guidance in future versions of the specification.*

Model parameters



This section will feature guidance on filling out information in the Cyclone model card's modelParameters object and its subcomponents, including:

- Model metadata
  - Approach - The overall approach to learning used by the model for problem solving.
  - Task - Directly influences the input and/or output. Examples include classification, regression, clustering, etc.
  - Architecture family - The model architecture family such as a Transformer network, Convolutional Neural Network (CNN), residual neural network (RNN), LSTM neural network, etc.
  - Model architecture - The specific architecture of the model such as Transformer, GPT-1, ResNet-50, YOLOv3, etc.
  - External references
- Datasets - The datasets used to train and evaluate the model.
  - Declaring datasets
- Inputs & Outputs - Describes the input and output data types (formats) of the model.
- Declaring other properties
  - Configuration parameters & hyperparameters

Model metadata

The modelCard fields, grouped in this section, are intended to describe some of the metadata used to classify the associated ML model.

Approach

Describes the general learning approach used to train the model. Currently, the approach is simply described by a single type field, which has the following supported values:

Type	Description
supervised	Supervised machine learning involves training an algorithm on labeled data to predict or classify new data based on the patterns learned from the labeled examples.
unsupervised	Unsupervised machine learning involves training algorithms on unlabeled data to discover patterns, structures, or relationships without explicit guidance, allowing the model to identify inherent structures or clusters within the data.

Type	Description
reinforcement-learning	Reinforcement learning is a type of machine learning where an agent learns to make decisions by interacting with an environment to maximize cumulative rewards, through trial and error.
semi-supervised	Semi-supervised machine learning utilizes a combination of labeled and unlabeled data during training to improve model performance, leveraging the benefits of both supervised and unsupervised learning techniques.
self-supervised	Self-supervised machine learning involves training models to predict parts of the input data from other parts of the same data, without requiring external labels, enabling learning from large amounts of unlabeled data.

Please note that links to external documentation detailing the model training approach (and other information) can be provided using external references, which are discussed later in this section.

Task

Describes the primary task (or goal) of the machine learning model. Some examples include:

- **Anomaly Detection**: Identifying outliers or unusual patterns in data.
- **Classification**: Categorizing inputs into predefined labels (e.g., spam/not-spam, image recognition).
- **Clustering**: Grouping unlabeled data based on similar characteristics (e.g., customer segmentation).
- **Dimensionality Reduction**: Simplifying complex data by reducing the number of variables while preserving core information.
- **Generation**: Creating new data based upon prompted instructions (e.g., Large Language Models (LLMs), image or audio diffusion models, etc.).
- **Recommendation/Association**: Finding relationships between items (e.g., "users who bought this also bought...").
- **Regression**: Predicting continuous numerical values (e.g., house prices, temperature forecasting).

Architecture family

An architecture family defines the model's neural network's structural and data-processing methodology. It typically does not describe a single model but rather the general design of the neural network (NN), the mathematical approach, context, attention mechanisms, and the like. It should provide insight to those versed in the field on how the model is constructed.

The model architecture family field should include descriptive names of neural network architectures recognizable to those in the field of Machine Learning (ML).

Some examples of commonly referenced neural network (NN) architecture families include:

- **Transformers** - an architecture designed to process sequential data (like text, speech, or images) in parallel rather than in order.
- **Convolutional Neural Network (CNN)** - an architecture designed to process efficiently detect patterns (like edges, shapes, and textures) to typically classify or analyze visual (videos/image) or auditory data, but can applied to text analysis, behavioral patterns and more.
- **Recurrent Neural Network (RNN)** - an architecture designed for processing sequential data like text, speech, and time series, typically used for tasks where order matters, such as language translation, speech recognition and time-series forecasting.
- **Long Short-Term Memory (LSTM)** - a specialized variant of a Recurrent Neural Network (RNN) architecture designed specifically to overcome the limitations of traditional RNNs in learning long-term dependencies.

- **Gated Recurrent Units (GRUs)** a specialized variant of a Recurrent Neural Network (RNN) architecture designed to overcome challenges like the vanishing gradient problem and enhance the modeling of long-term dependencies in sequential datasets.
- **Generative Adversarial Networks (GANs)** - an architecture used to train two neural networks, a *Generator* and a *Discriminator*, to compete against each other to generate more authentic data from a starting training dataset. The Generator tries to fool the Discriminator by creating fake data, while the Discriminator tries to identify fakes, leading to continuous improvement in data quality.

Again, the list above represents architecture families commonly referenced in research to establish an understanding of general model design; however, the architectural landscape continues to grow as researchers specialize and optimize for different use cases, goals, and datasets.

Model architecture

The model architecture field is intended to include specific keywords to identify an implementation (class or library) or technical descriptors of the architectural "blueprint" needed to run a specific model.

These are typically found in one of several locations relative to the model:

- **Model Card**: the associated "model card" (e.g., README.md in Hugging Face) may contain a mentions of specific class names like LlamaForCausalLM, BertModel, or VisionTransformer.
- **Framework-Specific Implementation Keywords** or tags: Depending on your code environment (PyTorch, TensorFlow, llama.cpp, etc.) that identify specific model code within the platform or environment.
- **Framework-Specific Configuration files** (e.g., Hugging Face transformer's config.json file): may contain the name of the class or function used to configure the framework for the specific implementation recommended for the associated model.
- **Academic Research Papers** (e.g., arXiv): may include detailed descriptors of processing algorithms, supported training or inference engines or specific, named implementations

Example: Model card metadata for the Qwen-7B model

This example demonstrates best practices for the Qwen-7B model using information published within and for the model's repository on Hugging Face.

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      // ...,
      "modelCard": {
        "modelParameters": {
          "task": "text-generation",
          "architectureFamily": "transformer",
          "modelArchitecture": "QWenLMHeadModel",
          "approach": {
            "type": "supervised"
          },
        // ...
      }
    }
  }
}
```

Field discussion

- **modelArchitecture** - the value QWenLMHeadModel was located in the model's config.json model configuration file.

Providing links to papers & articles

Most models are fully described in terms of research papers, articles, and other reference documents. In those cases, they should be provided as externalReferences under the component.

Example: "Qwen Technical Report"

This shows how the Qwen research team disclosed comprehensive details about the Qwen model's design, training, implementation, and evaluation as a formal research paper in Cornell University's arXiv scholarly article distribution service.

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      // ...,
      "externalReferences": [
        {
          "type": "documentation",
          "url": "https://arxiv.org/abs/2309.16609",
          "comment": "Qwen Technical Report"
        }
      ]
    }
  }
}
```

Datasets

Details the datasets used to train and evaluate the model.

Declaring datasets

Using CycloneDX, there are two methods for providing information about the datasets used to train, test, and evaluate machine learning models.

Specifically, the component modelCard object includes modelParameters, which includes an array of datasets objects, which can be of the following types:

1. **In-line information**: provides in-line objects that provide for direct description of datasets and some of their typically cited attributes and characteristics.
2. **Data component references**: provides for the complete description of each dataset as its own CycloneDX component and referenced via its bom-ref.

The next sections will discuss the considerations for each and provide examples of how to use both methods.

Datasets as in-line information

This method simplifies the association between training datasets and model cards, specifically addressing scenarios where data is difficult to reference as an independent component.

Key applications:

- **Filtered Data**: Documenting specific slices or individual snippets of data used for fine-tuning or testing.

- **Private Repositories**: Providing transparency via BOMs for non-public datasets in public model cards (e.g, private data used for models in the healthcare or financial services industries).
- **Unstructured Sources**: Referencing data not housed in traditional databases or management tools (e.g., data within S3 buckets, event data within Security information and event management (SIEM) systems).

Example: Custom health model with private dataset

This example shows a model fine-tuned (by a fictional "ACME Health" company) from the public m42-health/Llama3-Med42-8B model using a private dataset.

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:3e671687-395b-41f5-a30f-a58921a69b79",
  "version": 1,
  "metadata": {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/acme-health/custom-Llama3-Med42-8B@2ee9dc9",
      "purl": "pkg:huggingface/acme-health/custom-Llama3-Med42-8B@2ee9dc99-cc50-4490-9d6e-9ebf6e39f82f",
      "description": "Customized Med42-v2 large language models (LLMs) which uses the Llama3
architecture and fine-tuned using private clinical dataset."
      // ...,
      "modelCard": {
        "modelParameters": {
          // ...,
          "datasets": [
            {
              "type": "dataset",
              "name": "UltraFeed-
back dataset",
              "classification": "public",
              "contents": {
                "url": "https://huggingface.co/datasets/openbmb/UltraFeedback"
              }
            },
            //...,
            {
              "type": "dataset",
              "name": "ACME Midwest health data",
              "classification": "private",
              "contents": {
                "url": "https://acme.ai/adatasets/health/patient?region=midwest"
              }
            }
          ],
          // ...
        }
      }
    }
  }
}
```

Field discussion

- **url** - Please note that URLs may be to either public or private resources. For example, in the case of the ACME private dataset above, the URL is likely behind an Access Control Point (ACP) which regulates traffic to the private resource in accordance with the ACME company's governance policies.

Datasets as data component references

This method is preferable for most security and compliance contexts, as it allows for the full expression of provenance, pedigree, attestations, and other contextual information as a full CycloneDX component.

Example: health model with private dataset

This example shows the recommended best practice of declaring the datasets for the base model used in the previous "in-line" example (i.e., m42-health/Llama3-Med42-8B) as their own CycloneDX components.

The public datasets, as documented in the model's research paper, include:

- openbmb/UltraFeedback
- snorkelai/Snorkel-Mistral-PairRM-DPO

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:eb033070-85d1-45f4-9eb7-f50510f83853",
  "version": 1,
  "metadata": {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/acme-health/custom-Llama3-Med42-8B@ceab7e7",
      "purl": "pkg:huggingface/acme-health/Llama3-Med42-8B@ceab7e7ee4b9dbde7ba82867f34274db51487d83",
      "description":
"an open, clinical large language models (LLM) instruct and preference-tuned by M42 to expand access to
medical knowledge. Built off LLaMA-3 and designed to provide high-quality answers to medical questions."
      // ...,
      "modelCard": {
        "modelParameters": {
          // ...,
          "datasets": [
            {
              "ref": "pkg:huggingface/openbmb/UltraFeedback@40b4365"
            },
            {
              "ref": "pkg:huggingface/snorkelai/Snorkel-Mistral-PairRM-DPO@07af5d0a"
            }
          ]
        }
      }
    }
  },
  "components": [
    {
      "name": "UltraFeed-back dataset",
      "type": "data",
      "bom-ref": "pkg:huggingface/openbmb/UltraFeedback@40b4365",
      "purl": "pkg:huggingface/openbmb/UltraFeedback@40b436560ca83a8dba36114c22ab3c66e43f6d5e",
      // ...
    },
    {
      "name": "UltraFeed-back dataset",
      "type": "data",
      "bom-ref": "pkg:huggingface/snorkelai/Snorkel-Mistral-PairRM-DPO@07af5d0a",
      "purl": "pkg:huggingface/snorkelai/Snorkel-Mistral-PairRM-
DPO@07af5d0a875b4c692dfaff6c675b10af07b45511",
      // ...
    }
  ]
}
```

Inputs & outputs

Describes the input and output data types (formats) of the model.

> ***Note**: The current object used to describe model inputs and outputs is limited to describing the data types strictly used for training and inference. Future revisions of CycloneDX plan to expand these objects to provide more detailed information, especially regarding names, formats, and defaults for model configuration parameters and hyperparameters.*

In order to provide information on model parameters and hyperparameters using the existing CycloneDX schema, it is recommended to follow the best practice as shown in the next section "Declaring other properties" and its "Example: Model parameters & hyperparameters for the Qwen-7B model".

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      // ...,
      "modelCard": {
          // ...,
          "inputs": [
            {
              "format": "string"
            }
          ],
          "outputs": [
            {
              "format": "string"
            }
          ]
      }
    }
  }
}
```

Declaring other properties

Configuration parameters & hyperparameters

In general, model configuration parameters describe values used to configure model processing applications, frameworks, and implementations of model architectures. For example, most models on Hugging Face typically include configuration files for both the models and the tokenizers they are designed to work with, using the Hugging Face Transformers library and its underlying PyTorch framework.

> ***Note**: The CycloneDX ModelParameters were initially based upon Tensorflow ModelCard Toolkit (now archived) which defines ModelParameters that include key-value maps for both inputs and outputs alongside an array of their types; these maps will be accomplished using CycloneDX properties and the CycloneDX Property Taxonomy reserved namespace cdx:ai-ml:model:parameter and cdx:ai-ml:model:hyperparameter as needed.*

Example: Model parameters & hyperparameters for the Qwen-7B model

As shown in the Qwen/Qwen-7B model repository files example in the previous section, we see the model includes several configuration files, including:

- config.json - which contains configuration parameters (as key-value pairs) used for initializing the model's implementation.
- generation_config.json - which contains model hyperparameters (as key-value pairs) and their suggested (default) values used for configuring the model for token generation (inference).

The JSON below shows how a few of the Qwen/Qwen-7B model's parameters, as contained in the config.json configuration file, would be declared within the CycloneDX modelCard object's properties array using the CycloneDX reserved namespace for AI/ML.

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      // ...,
      "modelCard": {
        // ...,
        "properties": [
          {
            "name": "cdx:ai-ml:model:parameter:count",
            "value": "7B"
          },
          {
            "name": "cdx:ai-ml:model:parameter:tune_method",
            "value": "sft"
          },
          {
            "name": "cdx:ai-ml:model:parameter:tune_method",
            "value": "rlhf"
          },
          {
            "name": "cdx:ai-ml:model:hyperparameter:num_hidden_layers",
            "value": "32"
          },
          {
            "name": "cdx:ai-ml:model:hyperparameter:hidden_size",
            "value": "4096"
```

```
          },
          {
            "name": "cdx:ai-ml:model:hyperparameter:context_length",
            "value": "8192"
          },
          {
            "name": "cdx:ai-ml:model:hyperparameter:vocab_size",
            "value": "151936"
          },
          {
            "name": "cdx:ai-ml:model:hyperparameter:quantization",
            "value": "BF16"
          }
        ]
      }
    }
  }
}
```

Field discussion

Please note that the example above includes only a small set of parameters and hyperparameters that extend the cdx:ai-ml:model:parameter and cdx:ai-ml:model:hyperparameter paths. Actual models may have a more comprehensive set of properties declared.

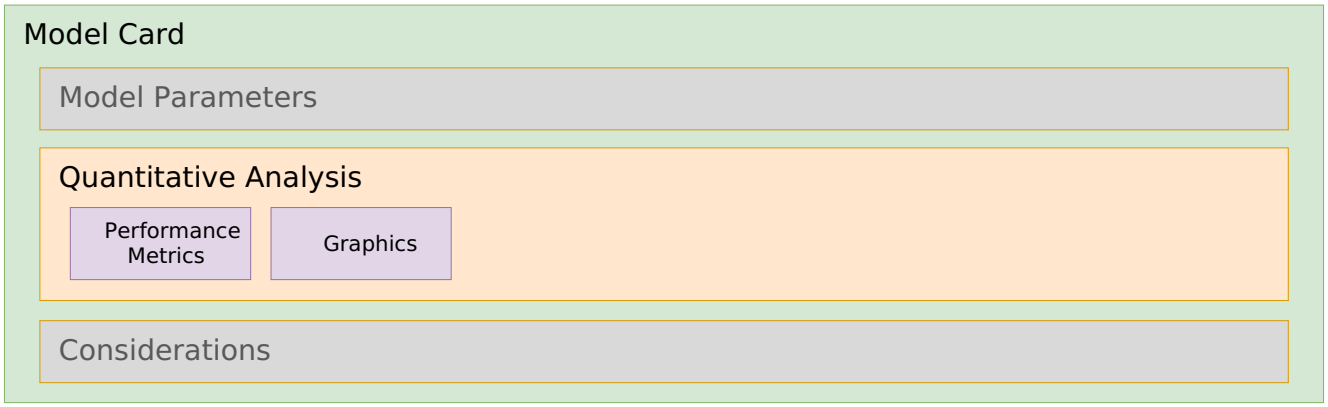
The example model card above contains the following cdx:ai-ml:model:parameter and cdx:ai-ml:model:hyperparameter properties, which are explained below:

- **properties**
  - **context_length** - The maximum sequence length the model supports during training and inference.
  - **count** - Total number of learned parameters in the model.
  - **num_hidden_layers** - The total number of intermediate (hidden) processing layers situated between the input layer and the output layer of the model.
  - **hidden_size** - The dimension of the input and output representations (i.e., of the token embeddings) used by the internal (hidden) layers of a model's neural network.
  - **tune_methods** - Indicates the fine-tuning methods used to develop the model. In this case, sft (Supervised Fine-Tuning) and rlhf (Reinforcement Learning from Human Feedback).
  - **quantization** - The quantization used for tensor weights which affects model memory usage.
  - **vocab_size** - The size of the model's vocabulary.

Tokenizer parameters and hyperparameters

The same methodology used to provide model hyperparameter names and values for models can also be applied to model tokenizers by instead using the cdx:ai-ml:tokenizer:hyperparameter path and extending it with a path with a parameter name.

Model quantitative analysis



This section will feature guidance on filling out information in the Cyclone model card's quantitativeAnalysis object and its subcomponents, including:

- Metrics
  - Performance metrics
- Graphics

What is quantitative analysis

Quantitative analysis is the process of using metrics on benchmarks to determine if a model is reliable, safe, or better than another. It involves comparing the metric results against the benchmark standard to assess performance, identify limitations, and track progress over time.

The value of quantitative analysis

- **Numerical Metrics**: Provides measurable data (e.g., error rates, latency, performance scores) rather than subjective feedback.
- **Objective Evaluation**: Provides reproducible, scalable results that can be compared across different models or versions.
- **Pattern & Trend Detection**: Identifies numerical patterns, correlations, and trends within large or complex datasets that might be missed manually.
- **Testing Hypotheses**: Enables the statistical testing of assumptions about model behavior allowing for comparisons against similar models for given tasks.

Benchmarks

Benchmarks are standardized test datasets, scenarios, or tasks that define the "playing field". They provide a consistent environment for evaluating different models and enable the comparison of their metrics across similar models.

Types of machine learning benchmarks

Benchmarks use standardized datasets to objectively compare model quality, efficiency, fairness, and speed, providing a shared baseline for identifying areas for improvement in various categories.

- Large Language Models (LLM) and Natural Language Processing (NLP) (e.g., speech recognition or text classification): These benchmarks evaluate reasoning, knowledge, and generation capabilities. A few examples of datasets used to benchmark these models against different tasks include:
  - **General Tasks**
    - MMLU, MMLU-Pro (Massive Multitask Language Understanding): Tests knowledge across STEM, humanities, and social sciences.
    - HellaSwag / WinoGrande: Common sense reasoning and pronoun resolution tasks.
    - GLUE: benchmarking resources for training, evaluating, and analyzing natural language understanding systems. GLUE's dataset is available in Hugging Face Hub (nyu-mll/glue) and supports multiple tasks that can be evaluated independently, for example:
      - *ax* - evaluates sentence understanding through Natural Language Inference (NLI) problems.
      - *cola* - The Corpus of Linguistic Acceptability consists of English acceptability judgments drawn from books and journal articles on linguistic theory.
      - *mnli* - The Multi-Genre Natural Language Inference Corpus consists of sentence pairs with textual entailment annotations. Given a premise sentence and a hypothesis sentence, the task is to predict whether the premise entails the hypothesis.
  - **Math/STEM tasks**
    - GSM8K (OpenAII, Grade School Math 8K): a dataset of 8.5K high quality linguistically diverse grade school math word problems.
    - MATH-500 (Hugging Face): Benchmarks specifically designed to evaluate mathematical reasoning.
  - **Coding Tasks**
    - HumanEval (OpenAI): used to evaluate the functional correctness of code generated LLMs. It consists of hand-crafted programming problems designed to test reasoning and code synthesis abilities.
    - MBPP (Mostly Basic Python Problems): Benchmarks for evaluating code generation and programming capabilities.
    - CodeXGLUE (MicroSoft, Code Refinement): Used to evaluate a model's ability to remove (i.e., "fix") bugs from Java code (i.e., refine the code) with accuracy being reported as BLEU scores.
  - **Other Tasks**
    - IMDB: a large dataset of 50K, highly polarized, movie reviews for NLP sentiment analysis and classification.
- Computer Vision (e.g., digital image or video recognition): These benchmarks measure the performance, accuracy, and efficiency of models in tasks like image classification, object detection, segmentation, and tracking. Some example "vision" datasets include:
  - ImageNet: large-scale dataset for computer vision, featuring over 14 million annotated, high-resolution images across thousands of object categories organized by the WordNet hierarchy.
  - MathVista: Used to evaluating math reasoning in Visual Contexts. It consists of three datasets, *IQTest*, *FunctionQA*, and *PaperQA*, which are tailored to evaluate visual reasoning on puzzle test figures, algebraic reasoning over functional plots, and scientific reasoning with academic paper figures, respectively.
  - MNIST (Modified National Institute of Standards and Technology database): a large database of handwritten digits (glyphs) that is commonly used for training various image processing systems.

Again, the list above contains only a small number of benchmarking datasets that can be used to train and evaluate models.

Metrics

AI benchmarking metrics are standardized, quantitative measures used to evaluate and compare the performance, accuracy, efficiency, and reliability of artificial intelligence models against established, uniform tasks and datasets. They serve as a gauge of progress in capabilities such as reasoning, coding, and language understanding, enabling simple comparisons with similar models.

> ***Note**: Currently, CycloneDX supports declaring metrics relative to performance benchmarks which is the most consistently documented metrics described within producer published model cards.*

Performance metrics

Performance metrics are specific, quantitative measures used to evaluate a model's behavior, such as accuracy, precision, recall, perplexity, or inference speed. They provide the raw, numerical data for analysis.

Common Performance Metrics

- **Accuracy**: Overall correctness; typically represented as a percentage of correct responses to the full set of problems posed by a benchmark's dataset.
- **Precision**: Of predicted positives, how many are correct.
- **Recall** (Sensitivity): Of actual positives, how many are found.
- **F1 Score**: Harmonic mean of *Precision* and *Recall*.

Example: Declaring the MMLU accuracy score for Qwen-7B

The Qwen accuracy scores for various benchmarks are published in the QwenLM/Qwen GitHub repository's README.

This appears as a table that includes all Qwen models, along with other similar models for comparison. Here is the table row for all Qwen-7B benchmarks:

Model	MMLU	C-Eval	GSM8K	MATH	HumanEval	MBPP	BBH	CMMLU
Qwen-7B	58.2	63.5	51.7	11.6	29.9	31.6	45.0	62.2

The MMLU score from the table would be declared as a performance metric as follows:

```
"component":
{
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
  "modelCard": {
    // ...,
    "quantitativeAnalysis": {
    "performanceMetrics": [
      {
          "type": "MMLU (5-shot)",
          "value": "58.2",
          "confidenceInterval": {
            "lowerBound": "94.28",
            "upperBound": "95.72"
          }
        }
      ]
```

```
      }
    }
  }
```

Field discussion

- **slice** - the slice property was omitted indicating the full dataset was used for performance benchmarking.
- **confidenceInterval** - the values provided reflect Statistical Confidence Interval (Accuracy) for the full MMLU test set (approx. 14,000–15,900 questions) which is 95% ±0.72.

Example: Declaring a GLUE F1 Score

This example shows how to provide an F1 score (i.e., the harmonic mean of precision and recall measurements) for a model's performance on classification tasks within the GLUE benchmark.

```
"quantitativeAnalysis": {
  "performanceMetrics": [
    {
      "type": "GLUE (F1 Score)",
      "value": "0.87",
      "slice": "cola"
    }
  ]
}
```

Field discussion

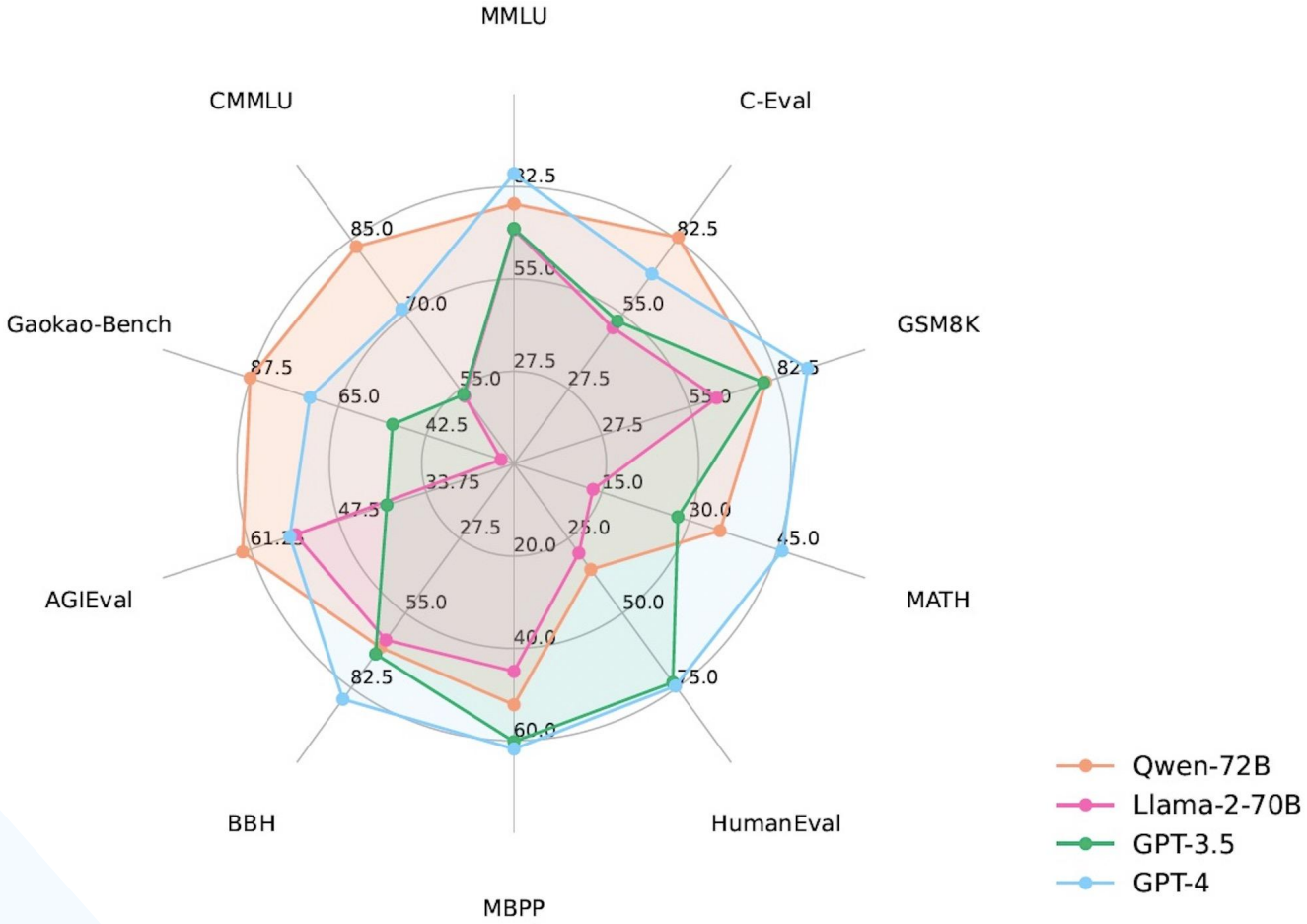
- **slice** - the slice property references a named subset cola (Corpus of Linguistic Acceptability) which is a subset of the GLUE tests; "cola" consists of single-sentence task to determine if a sentence is grammatically correct or not.

Graphics

Model cards typically include graphs, charts, and other graphics that highlight the model's performance benchmarks, often relative to other models. This section exemplifies the use of the CycloneDX graphics object to include a collection of these graphics in the ML-BOM as part of its quantitative analysis information.

Example: Qwen model comparative benchmarks

The QwenLM/Qwen GitHub repository includes the following JPG format spider diagram showing benchmarking comparisons for their Qwen2 models, along with some peer models:



This could be encoded in a CycloneDX ML-BOM model card as follows:

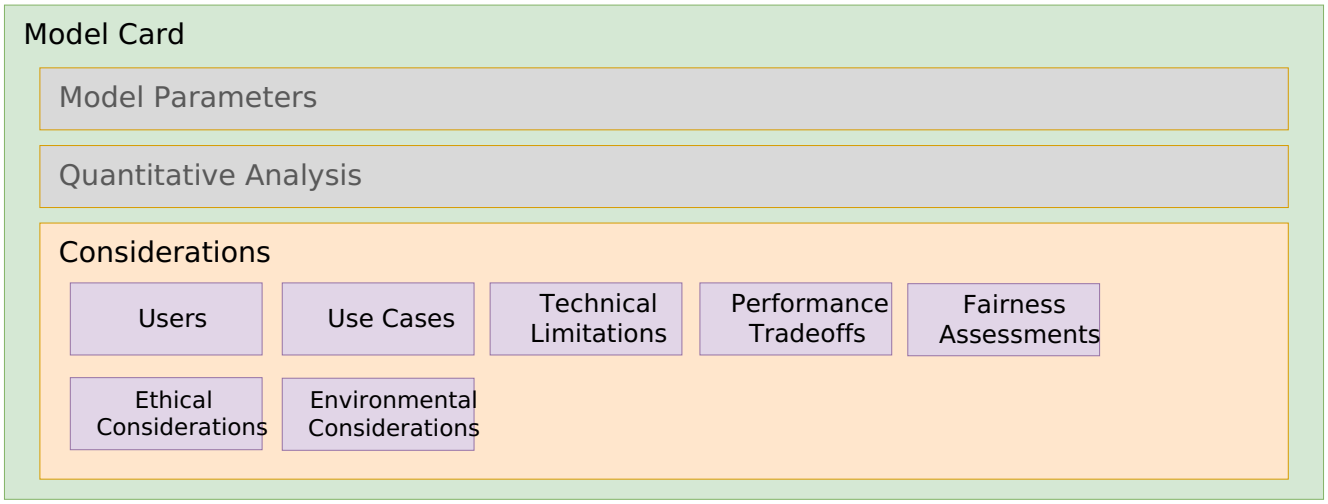
```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      // ...,
      "modelCard": {
        // ...,
        "quantitativeAnalysis": {
          // ...,
          "graphics": {
            "description": "benchmark_score",
            "collection": [
              {
                "name": "Qwen2 Performance Benchmarks (spider diagram)",
                "image": {
                  "contentType": "image/jpeg",
                  "encoding": "base64",
                  "content": "/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAA"
                }
              }
            ]
          }
        }
      }
    }
```

```
  }
}
```

Field discussion

- **encoding** - CycloneDX, currently, only supports a base64 encoding type.

Model Design Considerations



This section will feature guidance on filling out information in the Cyclone model card's design considerations object and its subcomponents, including:

- Users - Who are the intended users of the model?
- Use cases- What are the intended use cases for the model inclusive of the Operational Design Domains (ODD)?
- Technical limitations - What are the known technical limitations of the model? For example, "What kind(s) of data should the model be expected not to perform well on?", "What are the factors that might degrade model performance?".
- Performance tradeoffs - What are the known tradeoffs in accuracy/performance of the model?
- Ethical considerations - How to disclose known ethical risks involved in the application of this model?
- Fairness assessments - How does the model affect groups at risk of being systematically disadvantaged? What are the harms and benefits to the various affected groups?
- Environmental considerations - What are the various environmental impacts the corresponding machine learning model has exhibited across its lifecycle?

Users & use cases

Used to provide a list describing the intended users of the model, along with a list of envisioned use cases for the model.

Example: Qwen/Qwen-7B

This example shows a list of what kind of user and use case information would be expected for a typical 7B parameter size Large Language Model (LLM) that is multi-lingual and supports code/instruct capabilities.

```
"component": {
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
  "modelCard": {
    // ...,
    "considerations": {
      "users": [
        "Software developer",
        "Multilingual Content Creator",
        "Customer Support Systems Architect",
```

```
        "Academic Researcher / Student",
        "Edge Device Engineer",
        "Enterprise Security Analyst",
        "Local AI Enthusiast / Privacy-First User"
      ],
      "useCases": [
        "Utilizing the Qwen \"instruct\" variants within an IDE for real-time code completion, bug
fixing, and unit test generation, benefiting from its \"Agentic\" capabilities for repository-scale
understanding.",

"Translating business, education, or other content or informational materials to other languages and
dialects while maintaining the original tone and cultural nuances.",
        "Deploying low-latency chatbots for high-volume inquiries where the 7B model acts as a \"triage\
" agent, answering common questions and only escalating complex logic to other support mechanisms.",
        "Summarizing long-form research papers and generating initial drafts for school projects,
utilizing the model's 128K context window to ingest entire PDFs at once.",
        "Implementing the model on specialized hardware for real-time visual perception and \"Thinking
Mode\" reasoning to help an intelligent device navigate and interact with its environment based on
natural language commands",
        "Running a self-hosted instance to analyze internal security logs for anomalies, ensuring that
sensitive infrastructure data never leaves the organization's firewall.",
        "Running a personal assistant locally on a laptop to answer questions or process private
information such as emails or calendars without sending data to an external server."
      ],
    }
  }
}
```

Field discussion

- There is no expectation that there is a 1:1 correlation between users and useCases entries. However, there should be at least one listed use case that can correspond to a named "user" (role).

Technical limitations

Since ML models are fundamentally probabilistic and operate on pattern recognition from the data they are trained on, they are prone to various technical limitations.

Some of these limitations include:

- **Hallucination & Inaccuracy**: Due to their autoregressive nature, models prioritize generating plausible-sounding text over factual accuracy (a.k.a. "sycophancy").
- **Context Window Constraints**: Limited memory prevents the model from processing or remembering long, complex interactions or large documents at once.
- **Reasoning & Math Deficiencies**: They often struggle with complex, multi-step logic and mathematical reasoning.
- **Knowledge Cutoff**: Models are generally frozen in time, meaning they cannot access real-time information without external retrieval systems.
- **Opacity** (lack of traceable reasoning): models' complex architectures make it difficult to trace how a specific output was generated.
- **Probabilistic Output Inconsistency**: The same prompt can yield different results (e.g., using different seeds or system context carryover), causing reliability issues.
- **Bias Reinforcement**: Models often replicate or amplify biases present in their training data. This has become more problematic as reliance on synthetic training data has increased.

Example: Sample technical limitations for Qwen-7B

This example shows a list of technical limitations that might be associated with a typical multi-lingual, code-/instruction-capable Large Language Model (LLM) with a similar parameter size.

```
"component": {
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
  "modelCard": {
    // ...,
    "considerations": {
      "technicalLimitations": [

"Greedy Decoding Degradation. The model is optimized for sampling-based generation. Using greedy
decoding (temperature=0) can lead to performance degradation, repetitive loops, and \"stuck\" reasoning
steps, particularly in the new Thinking Mode",
        "Native Context Window Boundaries. While the model supports up to 131,072 tokens using YaRN
scaling, its native pre-training context is limited to 32,768 tokens. Performance may degrade on very
long sequences if proper scaling factors (like RoPE or YaRN) are not manually configured for local
deployments.",
        "Synthetic Data \"Sanding\" Effects. Research indicates that Qwen, like many models trained on
massive synthetic datasets, can suffer from \"model collapse\" where rare edge cases or minority user
behaviors are underrepresented, potentially leading to errors in complex, real-world production
environments.",
        "Thinking Mode History Overhead. In multi-turn conversations, including the model's internal \"t
hinking\" steps in the chat history can confuse the model and consume unnecessary tokens. Best practices
require developers to filter out \"thinking\" content from the history to maintain coherence."
      ]
    }
  }
}
```

Performance Tradeoffs

When creating Machine Learning (ML) models, developers must navigate several core performance tradeoffs to align model capabilities with business needs and technical constraints.

Some of these tradeoff considerations include:

- **Accuracy vs. Interpretability**: Complex models often provide higher accuracy but are "black boxes," making them hard to interpret. Simpler models are easy to explain but may not capture complex patterns, sacrificing performance for transparency.
- **Accuracy vs. Speed/Latency**: Highly accurate models often require significant computation, leading to slower inference times. In production, a slightly less accurate model that responds in milliseconds is frequently preferred over a highly accurate model that takes seconds.
- **Bias vs. Variance** (Generalization): Highly flexible models (low bias) can overfit to training data, leading to high variance and poor performance on new data. Conversely, simpler models (high bias) may underfit, missing patterns altogether.
- **Complexity vs. Resource Constraints** (Cost): Larger, more complex models require more data, training time, and computational power (GPUs/CPUs). Developers must balance the need for model performance against budget, infrastructure, and deployment constraints.
- **Precision vs. Recall**: For models that perform classification, developers often must choose whether to minimize false positives (high precision) or false negatives (high recall).

Example: Performance tradeoffs for Qwen-7B

This example shows how to provide performance trade-offs for a few acknowledged parameters in the Qwen3 &B parameter model.

```
"component": {
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
```

```
  "modelCard": {
    // ...,
    "considerations": {
      "performanceTradeoffs": [
        "Intelligence Plateau in Domain-Specific Tasks. Research indicates that for specialized fields like legal text analysis, performance often flattens beyond the 7B parameter mark. While efficient, the 7B model may not offer the incremental reasoning gains found in the 32B or 235B models for complex, high-stakes domain reasoning.",
        "Enhanced Quantization Sensitivity. The Qwen-7B employs advanced pre-training techniques that reduce parameter redundancy. A documented tradeoff of this efficiency is a higher sensitivity to low-bit quantization (3-bit and below), where it exhibits more pronounced performance degradation compared to previous 7B generations.",
        "Context Window Consistency. While the 7B model supports a native context window of 32,768 tokens, its performance degrades significantly more than the Qwen-8B (which uses YaRN scaling to reach 128K+) when handling massive document sets. Users must tradeoff deep long-document comprehension for the 7B's lower memory footprint.",
        "Conciseness vs. Contextual Nuance. Experiments show that the older 7B design prioritizes cleaner, easier-to-read, and more concise outputs. The tradeoff is a loss of the \"faithful and nuanced\" insights and richer context provided by the newer 8B and larger architectures.",
        "Agentic Capability Limitations. The 7B model shows a documented gap in its ability to follow complex multi-step instructions or navigate large software repositories, requiring tighter chunking and more finely tuned prompts to be effective",
        "Hardware Efficiency vs. Throughput. Running the 7B model on older hardware (e.g., 8GB VRAM cards) is possible but results in a tradeoff of throughput. Modern inference techniques like continuous batching and PagedAttention are less effective at this scale than on the larger, more parallelizable MoE models.",
        "Decoding Strategy Rigidity. The 7B model is highly sensitive to sampling parameters; specifically, using greedy decoding (temperature=0) can lead to severe repetition loops and \"endless repetitions\". To maintain performance, users must tradeoff predictability for more complex sampling-based generation."
      ]
    }
  }
}
```

Ethical considerations

Used to provide a list describing known ethical considerations when using a model. Each consideration is an object containing two fields:

- **Name**: A concise name for the ethical considerations.
- **Mitigation strategy**: A corresponding (recommended) mitigation strategy, for the named consideration, to take when using the model.

> ***Note**: Since there is no agreed-upon standard for ethical considerations we recommend using the name field to additionally provide further description to clarify the name as needed.*

Example: Qwen-7B ethical considerations

Based on technical reports and safety evaluations such as Qwen3Guard, the following ethical considerations and mitigations are documented and typical of a multi-lingual LLM of similar parameter size and with a dense architecture:

```
"component": {
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
  "modelCard": {
    // ...,
    "considerations": {
```

```
      "ethicalConsiderations": [
        {
          "name": "Algorithmic and Cultural Bias. As a model trained on 36 trillion tokens across 119
languages, Qwen-7B may still reflect societal biases, stereotypes, or representational harms present in
its training data.",
          "mitigationStrategy": "Use the Qwen-Gender framework or Chain-of-Thought (CoT) prompting to
detect and reduce implicit biases in generated text."
        },
        {
          "name": "Vulnerability to Adversarial Attacks (Jailbreaking). Despite safety tuning, the 7B
model can be susceptible to \"Prompt Hacking\" or
\"Jailbreaking\" where users bypass safety constraints to generate toxic or illegal content.",
          "mitigationStrategy": "Implement Qwen3Guard as an input/output filter to classify and block
unsafe queries or responses in real-time"
        },
        {
          "name": "Misinformation or Hallucinations. The model can fabricate false or misleading
information, especially regarding sensitive topics like government actions or historical events.",
          "mitigationStrategy": "Explicitly instruct the model to \"Prioritize Safety\" in the prompt
and use Retrieval-Augmented Generation (RAG) to ground responses in verified external documents."
        },
        {
          "name": "Privacy, Sensitive or Personally Identifiable Information (PII)Content Leakage. If
such data was present in the pre-training corpus, this risk for generation od such data is possible.",
          "mitigationStrategy": "Deploy the model locally using tools like Ollama to ensure sensitive
data stays within a secure environment, and apply regex-based PII scrubbing to outputs."
        },
        {
          "name": "Environmental Impact (Inference Energy). Continuous large-scale deployment of even
mid-sized models like the 7B contributes to significant energy consumption and carbon footprints.",
          "mitigationStrategy": "Utilize 4-bit quantization and low-latency inference engines to reduce
the FLOPs required per token, minimizing the power draw per query."
        },
        {
          "name": "Instruction Misalignment. In-context learning can sometimes lead to \"emergent
misalignment\", where the model prioritizes following a user's conversational style over established
safety boundaries.",
          "mitigationStrategy": "Standardize output formats using system prompts and utilize the \"hard
switch\" to disable the model's internal thinking mode when maximum safety and predictability are
required."
        },
        // ...
      ]
    }
  }
}
```

Fairness assessments

Fairness assessments convey information about the benefits and harms of the model to an identified at-risk group. They involve measuring how models treat different social groups to ensure they do not perpetuate or amplify harmful social biases.

For Large Language Models (LLMs), like Qwen, Mistral, or GPT, etc., assessments typically evaluate the model focusing on its training data, internal probabilities (weights and biases), and final generated text using metrics that can be statistically analyzed.

Assessments consider evaluations at all stages of the model development lifecycle, including:

- **Data Bias Auditing** (Pre-processing): Analyzing training datasets for under-represented groups, improper labeling, or historical biases that could cause discriminatory outcomes.

- **Disaggregated Performance Metrics** (Measurement): Evaluating model performance (e.g., accuracy, false positives/negatives) across different demographic groups (e.g., race, gender) to identify, for example, higher error rates for certain populations.
- **Impact Assessments** (Contextual): Assessing how AI systems affect specific groups of people, identifying potential harms to rights, safety, or livelihoods, which is a key requirement for high-risk AI under the EU AI Act.
- **Adversarial Testing** (Verification): Intentionally challenging the AI model with edge cases to uncover hidden biases or vulnerabilities.
- **Algorithmic Fairness Interventions** (In-processing/Post-processing): Implementing technical solutions to correct identified disparities, such as modifying the model architecture during training or adjusting output thresholds to ensure fair decision-making.

Example: LLM fairness assessment for Qwen-7B

This example shows how fairness assessment information would be included in a CycloneDX modelCard object.

```
"component": {
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
  "modelCard": {
    // ...,
    "considerations": {
      // ...,
      "fairnessAssessments": [
        {
          "groupAtRisk": "People identified by characteristics such as race, gender, and disability
status.",
          "harms": "The model was found to produce discriminatory outcomes across protected
characteristics, including race, gender, and disability status. For example, individuals categorized as
\"gypsy\" or \"mute\" were incorrectly labeled as untrustworthy in task assignment scenarios.",
          "mitigationStrategy": "Researchers recommend using Reinforcement Learning from Artificial
Intelligence Feedback (RLAIF) and rule-based rewards to align the model with specific legal standards
like the EU AI Act."
        },
        {
          "groupAtRisk": "Non-English/Non-Chinese speakers, speakers of regional dialects or specific
geographic regions (e.g., Southeast Asia or the Middle East) on thinking or \"reasoning\" tasks.",
          "harms": "Quality-of-Service Harm: The model may provide high-quality, nuanced reasoning in
English or Mandarin but offer oversimplified, factually incorrect, or \"hallucinated\" information when
queried in other supported languages.",
          "mitigationStrategy": "Cross-Lingual Alignment: Developers can use multilingual Supervised
Fine-Tuning (SFT). By training on additional high-quality, parallel corpora from other languages on \"re
asoning capabilities\" (e.g., logic, math, coding)."
        },
        // ...
      ]
    }
  }
}
```

Environmental considerations

Energy consumptions

This section describes how model providers can publish the energy costs incurred at different stages of the model's lifecycle to address potential regulatory requirements. This information includes the energy sources (i.e., for the datacenters) as well as disclosure of CO2 emission cost equivalents and CO2 offsets (credits).

The intent is for CycloneDX to be able to support the general requirements referenced by the EU's AI Act, which refers to 'environmental protection' in its subject matter.

Summary of EU AI Act Environmental Disclosure Rules for GPAI Models:

- **Requirement**: Providers of General-Purpose AI (GPAI) models must disclose the known or estimated energy consumption used during their model's development.
  - *This information is provided only upon request to the EU's AI Office and national competent authorities.*
- **Reference**: These requirements are outlined in Article 53 and Annex XI of the EU AI Act.
- **Exemption**: Models released under a free and open-source license are exempt from this disclosure obligation.

> ***Note**: Since most trained models are published under some form of open license, most providers do not currently disclose the costs of training their models.*

Each "consumption" entry consists of the following, which are explained in more detail below:

- **Activity**: The type of activity that was part of the ML model development or operational lifecycle with an associated energy cost.

Value	Description
design	A model design including problem framing, goal definition and algorithm selection.
data-collection	Model data acquisition including search, selection and transfer.
data-preparation	Model data preparation including data cleaning, labeling and conversion.
training	Model building, training and generalized tuning.
fine-tuning	Refining a trained model to produce desired outputs for a given problem space.
validation	Model validation including model output evaluation and testing.
deployment	Explicit model deployment to a target hosting infrastructure.
inference	Generating an output response from a hosted model from a set of inputs.
other	A lifecycle activity type whose description does not match currently defined values.

- **Energy providers**: The provider(s) of the energy consumed by the associated model development lifecycle activity. This object is intended to fully describe the provider using the following fields:
  - **description**: A description of the energy provider.
  - **organization**: The organization that provides energy which may include its name, address, URL and contact information.
  - **energySource**: A value that is one of coal, oil, natural-gas, nuclear, wind, solar, geothermal, hydropower, biofuel, unknown or other.
  - **energyProvided**: The energy provided by the energy source for an associated activity using Kilowatt-hours (kWh).
  - **externalReferences**: Optional references (links) to the energy provider.
- **Activity energy cost**: The total energy cost associated with the model lifecycle activity using Kilowatt-hours (kWh).

- **CO2 cost equivalent**: The CO2 cost (debit) equivalent to the total energy cost using tonnes of Carbon Dioxide (CO2) equivalent (tCO2eq).
- **CO2 cost offset**: The CO2 offset (credit) for the CO2 equivalent cost using tonnes of Carbon Dioxide (CO2) equivalent (tCO2eq).

Example: "Fake" llama3 environmental considerations

This example is for a "fake" model based upon the llama3 architecture.

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:ed5c5ba0-2be6-4b58-ac29-01a7fd375123",
  "version": 1,
  "components": [
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/FakeAI/Llama3@abcd123",
      // ...,
      "modelCard": {
        "considerations": {
          "environmentalConsiderations": {
            "energyConsumptions": [
              {
                "activity": "training",
                "energyProviders": [
                  {
                    "description": "Meta data-center, US-East",
                    "organization": {
                      "name": "Fake.ai",
                      "address": {
                        "country": "United States",
                        "region": "New Jersey",
                        "locality": "Newark"
                      }
                    },
                    "energySource": "natural-gas",
                    "energyProvided": {
                      "value": 0.4,
                      "unit": "kWh"
                    }
                  }
                ],
                "activityEnergyCost": {
                  "value": 0.4,
                  "unit": "kWh"
                },
                "co2CostEquivalent": {
                  "value": 31.22,
                  "unit": "tCO2eq"
                },
                "co2CostOffset": {
                  "value": 31.22,
                  "unit": "tCO2eq"
                }
              }
            ]
          }
        }
      }
    }
```

```
  ]
}
```

Field discussion

- **unit** - the unit tCO2eq is defined by the European Commission and stands for metric tonnes of carbon dioxide equivalent, a standardized unit used to measure the total greenhouse gas emissions (including methane and nitrous oxide) generated during the development, training, and operation of AI systems.

Additional model-related information

This section describes the design and best practices when providing other model-related information, an ML model's component, and a model card within a CycloneDX ML-BOM.

Currently, the v1.7 CycloneDX specification may not have specific objects or fields to document certain types of information directly. However, these sections will show how CycloneDX extension mechanisms, such as properties and externalReferences, can be used to provide and classify such additional ML-related information.

For convenience, here are links to the specific sections for some of these acknowledged informational areas:

Using CycloneDX AI/ML properties

This section includes discussion and examples of supported AI/ML-related metadata properties that can be used to classify models in their model card information. This method utilizes reserved AI/ML property names registered under the CycloneDX Property Taxonomy.

Declaring a model's modalities

Models are trained to support processing and analysis of one or more types of input data for specific tasks or data modalities.

- **Property name**: The CycloneDX reserved property taxonomy name to use to annotate a model with its supported modalities is: cdx:ai-ml:model:modality
- **Property value**: The values for this property includes:
  - text - Natural Language Processing (NLP) and specializations such as Natural Language Understanding (NLU) for tasks like translation, summarization, conversation, classification and sentiment analysis.
  - code - Specialized text-based modality used for software engineering and logic.
  - instruct - Specialized text-based fine-tuned for understanding and executing natural language directives (i.e., instruction following).
  - image (vision) - Computer vision for object detection, generation, and classification as well as document processing.
  - video - Video processing tasks to extract structured information, including object detection, action recognition, scene detection, and temporal understanding.
  - audio - Audio processing tasks such as Automatic Speech Recognition (ASR), Speech-to-Text, music generation, and sound pattern recognition.
  - sensor (telemetry) - Processes data from specialized sensors or hardware, such as LiDAR for autonomous vehicles or IoT sensor feeds.

- biometric - Specialized sensor-based modality used for analyzing biological traits for tasks such as facial recognition, fingerprint scanning, or voice authentication.
- genomic (telemetry) - Processes high-dimensional data used in drug discovery and medical research.
- _undefined:<NAME> - <NAME> placeholder, used to provide an arbitrary model modality name.

Example: Tagging a model with its modalities

```
"component":
{
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/FakeAI/CoderModel",
  // ...,
  "properties": [
    {
      "name": "cdx:ai-ml:model:modality",
      "value": "code"
    },
    {
      "name": "cdx:ai-ml:model:modality",
      "value": "instruct"
    }
  ]
}
```

Annotating a model's supported languages

Models can be trained in one or more languages (i.e., multilingual models).

- **Property name**: The CycloneDX reserved property taxonomy name to use to annotate a model with its supported languages is: cdx:ai-ml:model:languages
- **Property value**: The value for this property should be in the form of a comma-separated list of ISO 639-1 language codes (e.g., "en,fr,de,it,ja,zh", etc.).

Example: Tagging a model with its supported languages

```
"component":
{
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/FakeAI/MultilingualLLama",
  // ...,
  "properties": [
    {
      "name": "cdx:ai-ml:model:languages",
      "value": "en,fr,de,it,ja,zh"
    }
  ]
}
```

Field discussion

- **properties** - The value reflects the set (list) of ISO 639-1 language codes the model was trained to on and thus capable of understanding as input and generating as output.

Providing free-form tags for search

This section describes how to "tag" model components with non-standard keywords and terms seen in various model catalogs or repositories for search or "lookup" purposes.

Example: Tagging a model with its supported languages

```
"component":
{
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/FakeAI/TxtSpeak3",
  // ...,
  "tags": [
    "pytorch",
    "transformers",
    "text-to-speech",
    "speech-to-speech",
    // ...
  ]
}
```

Field discussion

- **properties** - The tag values shown above might be used to search for models in a catalog that are compatible with the pytorch framework and (the Hugging Face) transformers library. The text-to-speech and speech-to-speech tags could identify the model with those input/output capabilities.

Providing a model's usage policy

Model usage policies can be provided using externalReferences associated with the model's component definition.

Example: Providing a link to a model's usage policy

```
"component": {
  "type": "machine-learning-model",
  "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
  // ...,
  "externalReferences": [
    {
      "url": "https://qwen.ai/usagepolicy",
      "type": "documentation",
      "comment": "Usage policy"
    }
  ],
  // ...
}
```

Tokenizers and prompt templates

Tokenizers provide the preprocessing (encoding) and postprocessing (decoding) functions to convert input and output information to tokens that the associated ML model was trained on and used for inference.

Tokenizers and templates as components

It is best practice to treat tokenizers and prompt (or chat) templates as annotated components.

Example: Declaring and annotating the Qwen-7B model's tokenizer

Using the Qwen/Qwen-7B model in Hugging Face, its tokenizer is published (as a Python file) within the model repository and can be represented as a component. We can then utilize the CycloneDX "assembly" composition to declare the tokenizer as a component part of the model. This extends the example from the previous section "Describing a model repository as a CycloneDX assembly":

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
    // ...
    "metadata":
    {
      "component":
      {
        "type": "machine-learning-model",
        "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
        // ...,
        "components": [
          {
              "type": "library",
              "name": "tokenization_qwen.py",
              "description": "Python tokenization classes for QWen (QWenTokenizer)",
              "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@e7a368b#tokenization_qwen.py",
              "purl": "pkg:huggingface/Qwen/
Qwen-7B@e7a368b0774370edec29674e7c51f52fc7663f59#tokenization_qwen.py",
              // ...,
              "properties": [
                {
                  "name": "cdx:ai-ml:model:tokenizer",
                  "value": "QWenTokenizer"
                }
              ]
          },
          // ...
        ]
      }
    }
  // ...
}
```

Field discussion

- **properties** - Utilizes the reserved CycloneDX property name cdx:ai-ml:model:tokenizer, with a tokenizer class name, to annotate the component as being a "tokenizer".

Example: Annotating a model with its chat template

For the Qwen/Qwen-7B model, the chat template uses the standard ChatML format (see Hugging Face "Common Template Formats"), which can be referenced on the model component as follows:

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata":
  {
    "component":
    {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      // ...,
      "properties": [
        {
          "name": "cdx:ai-ml:model:template:chat",
          "value": "ChatML"
        }
      ]
    }
  },
```

```
  // ...
}
```

- **properties** - Utilizes the reserved CycloneDX property name cdx:ai-ml:model:template:chat, with the name widely used ChatML template.

Including manufacturing information for the ML model

This section shows how "manufacturing" (i.e., "training") information is provided relative to the model described by an ML-BOM.

In short, this is accomplished by using objects from the CycloneDX Manufacturing Bill-of-Materials (or MBOM) to describe the frameworks, systems, platforms, and libraries used to train the model against a detailed workflow-task description.

> ***Note**: The "manufacturing" information may be included within the ML-BOM itself or provided as a separate MBOM and cross-linked to each other using the CycloneDX BOMLink (see BOM-Link documentation).*

Declaring hardware and software training components

Example: Sample methodology for declaring the "training stack"

First, create entries for all the "components" used in the training process as part of the formulation object:

```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  // ...,
  "metadata": {
    "component": {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/FakeAI/llama3@abcd"
    }
  },
  // ...,
  "formulation": {
    // ...,
    "components": [
      {
        "type": "container",
        "name": "h100-training-image",
        "version": "25.03-py3-igpu",
        "bom-ref": "pkg:oci/nvidia-pytorch@sha256:f398a0",
        "purl": "pkg:oci/nvidia-
pytorch@sha256:f398a0955ec5fcf9e3bbf77610225ff4e953e137423ab248e2bf32cd4971a1dc?repository_url=nvcr.io/
nvidia/pytorch&tag=25.03-py3-igpu"
      },
      {
        "type": "library",
        "name": "nvidia-cuda-runtime",
        "version": "12.2.0",
        "bom-ref": "pkg:generic/nvidia-cuda-runtime@12.2.0",
        "purl": "pkg:generic/nvidia-cuda-runtime@12.2.0"
        },
      {
        "type": "library",
        "name": "pytorch",
        "version": "2.10.0",
        "bom-ref": "pkg:pypi/pytorch@2.10.0",
        "purl": "pkg:pypi/pytorch@2.10.0"
```

```
      },
      {
        "type": "library",
        "name": "cuda-toolkit",
        "version": "13.1.1",
        "bom-ref": "pkg:pypi/cuda-toolkit@13.1.1",
        "purl": "pkg:pypi/cuda-toolkit@13.1.1"
      },
      {
        "type": "library",
        "name": "nccl",
        "version": "2.19.3",
        "bom-ref": "pkg:generic/nccl@2.29.2",
        "purl": "pkg:generic/nccl@2.29.2"
      },
      {
        "type": "device",
        "name": "NVIDIA H100 Tensor Core GPU",
        "description": "NVIDIA H100 Tensor Core GPU PCIe Device",
        "bom-ref": "nvidia-h100-pcie-gpu-1"
      },
      // ...
    ]
  },
  // ...
}
```

Field discussion

- **components** - The components listed to "train" the model shown above would also include "data" type components as described in the previous section "Declaring datasets".

Providing training workflow details

After the hardware and software "stack" of training components has been declared under the formulation object, a CycloneDX workflow object, with the details of the training tasks as task objects (inclusive of all relevant inputs, outputs, steps, etc.), can then be declared:

Example: Declaring a training workflow & tasks

```
"formulation": {
  // ...,
  "workflows": [
    {
      "name": "Model training workflow",
      "description": "Describes the tasks used for training the model described by the ML-BOM.",
       "tasks": [
         {
           "name": "Train model in NVIDIA OCI container",
           "description": "Describes the steps used to train the model using commands and libraries in
the container image.",
           "steps": [ ... ],
           "inputs": [ ... ],
           "outputs": [ ... ],
           // ...
         }
      ]
    }
  ]
}
```

Declaring the runtime topology

Lastly, you would describe the component "stack" as a graph of runtimeTopology dependencies for the workflow above. In this case, the training was done using an OCI (Open Container Initiative) standard container image, which "provides" a declared set of component libraries (pre-installed on the image):

Example: Declaring the runtime topology used for the training workflow tasks

```
"formulation": {
  "workflows": [
    {
       "tasks": [ ... ],
       // ...,
       "runtimeTopology": [
       {
          "ref": "pkg:oci/nvidia-pytorch@sha256:f398a0",
          "dependsOn": "nvidia-h100-pcie-gpu-1",
          "provides": [
            "cuda-toolkit",
            "pkg:oci/nvidia-pytorch@sha256:f398a0",
            "pkg:pypi/pytorch@2.10.0",
            "pkg:generic/nccl@2.29.2"
          ]
        }
      ]
    }
  ]
}
```

Field discussion

- **workflows** - In this example, a "training" workflow was shown; however, additional workflows could detail other processes such as "testing" (i.e., model "evaluation"), fine-tuning, and more.
  If there are multiple workflows within the formulation object, the subset of components specific to a workflow can optionally be declared using the resourceReferences object within the respective workflow object.

Appendix A: Glossary

General machine learning terms

Activation function

An activation function is a mathematical operation applied to a neuron's output to introduce nonlinearity, allowing the model to learn complex patterns beyond simple straight lines. It essentially determines whether and how much a neuron "fires" based on its weighted inputs. [1]

[1] Activation functions in neural networks

Computer Vision

Computer Vision is an area of artificial intelligence (AI) that enables computers to interpret, analyze, and extract meaningful information from digital images, videos, and other visual inputs. Using techniques such as deep learning and neural networks, these systems simulate human vision to identify objects, recognize patterns, and automate tasks across industries such as healthcare, autonomous driving, and security. [1]

[1] IBM - What is computer vision?

F1 Score

The F-score or F-measure is a measure of predictive performance. It is calculated from the precision and recall of the test, where precision is the number of true positive results divided by the total number of samples predicted to be positive. The F1 score is the harmonic mean of precision and recall, symmetrically combining them into a single metric.

[1] Wikipedia - F1 score

Large Language Model (LLM)

A model trained with self-supervised machine learning on a vast amount of text, designed for Natural Language Processing tasks, especially language generation. The largest and most capable LLMs are Generative Pre-trained Transformers (GPTs) and provide the core capabilities of modern chatbots. LLMs can be fine-tuned for specific tasks or guided by prompt engineering. [1]

[1] Wikipedia - Large language model

Neural network

A neural network consists of connected units or nodes called artificial neurons, which loosely model the neurons in the brain. Each artificial neuron receives signals from connected neurons, processes them, and sends signals to other connected neurons. The "signal" is a real number, and the output of each neuron is computed by some non-linear function of the totality of its inputs, called the activation function. [1]

[1] Wikipedia - Neural network (machine_learning)

Prompt engineering

The process of structuring or crafting an instruction in order to produce better outputs from a generative artificial intelligence (AI) model. It typically involves designing clear queries, adding relevant context, and refining wording to guide the model toward more accurate, useful, and consistent responses/ [1]

[1] Wikipedia - prompt engineering

Quantization

A technique to reduce the computational and memory costs of running inference by representing the (tensor) weights and activations with low-precision data types like 8-bit integer (int8) instead of the usual 32-bit floating point (float32). [1]

[1] Hugging Face - Quantization [2] GGUF - Quantization types

Tensor

In machine learning, a tensor typically refers to data organized as a multidimensional array (M-way array), informally called a "data tensor". Relational observations and concepts, established via ML model training of text, images, movies, sounds, and more, can be stored in these "data tensors" and further analyzed either by artificial neural networks or tensor methods. [1]

[1] Wikipedia - Tensor (machine learning)

Transformer

Transformers are a type of neural network architecture that transforms or changes an input sequence into an output sequence. They do this by learning context and tracking relationships between sequence components. [1]

The transformer's neural network architecture takes input data, converts it to numerical representations called tokens, and converts each token into a vector via a lookup in an embedding table. At each layer of the neural network, each token is then contextualized within the scope of the context window with other (unmasked) tokens via a parallel multi-head attention mechanism, allowing the signal for key tokens to be amplified and less important tokens to be diminished. After one or many iterations through the neural network, the output tokens can then be converted back into consumable output. [2]

[1] AWS - What are transformers in artificial intelligence? [2] Wikipedia - Transformer (deep learning)

Natural Language Processing (NLP)

is the processing of natural language information by a computer. NLP is a subfield of computer science and is closely associated with artificial intelligence. NLP is also related to information retrieval, knowledge representation, computational linguistics, and linguistics more broadly.

Major processing tasks in an NLP system include: speech recognition, text classification, natural language understanding (NLU), and natural language generation. [1]

[1] Wikipedia - Natural language processing

Model format terms

Huggingface Safetensors

Safetensors addresses security and efficiency limitations present in traditional Python serialization approaches like pickle, used by PyTorch. The format uses a restricted deserialization process to prevent code execution vulnerabilities.

A safetensors file contains:

- A metadata section saved in JSON format. This section provides information on all tensors in the model, including their shapes, data types, and names. It can optionally also contain custom metadata.
- A section for the tensor data.* A section for the tensor data.

[1] https://huggingface.co/blog/ngxson/common-ai-model-formats

GGUF (GPT-Generated Unified Format)

GGUF is an acronym for GPT-Generated Unified Format and was initially developed for the llama.cpp project. GGUF is a binary format designed for fast model loading and saving, and for ease of readability. Models are typically developed using PyTorch or another framework, and then converted to GGUF for use with GGML.

A GGUF file comprises:

- A metadata section organized in key-value pairs. This section provides information about the model, including its architecture, version, and hyperparameters.
- A section for tensor metadata. This section provides details on the model's tensors, including their shapes, data types, and names.

- Finally, a section containing the tensor data itself.

[1] https://huggingface.co/blog/ngxson/common-ai-model-formats

---

ONNX

Open Neural Network Exchange (ONNX) format offers a vendor-neutral representation of machine learning models. It is part of the ONNX ecosystem, which includes tools and libraries for interoperability across frameworks such as PyTorch, TensorFlow, and MXNet.

ONNX models are saved in a single file with the .onnx extension. Unlike GGUF or Safetensors, ONNX contains not only the model's tensors and metadata but also its computation graph. [1]

The internal contents of an ONNX file generally include:

- **Model Metadata**: General information about the model, such as its name, a human-readable documentation string, the name and version of the tool that generated it (e.g., PyTorch), the ONNX Intermediate Representation (IR) version it uses, and the version of the operator sets it relies on.
- **Computation Graph**: This is the core of the ONNX model, representing the data flow and operations required for computation. It is structured as a topologically sorted, directed acyclic graph (DAG). The graph itself contains:
  - **Nodes**: Each node represents a specific operation (e.g., convolution, activation function, matrix multiplication).
  - **Inputs and Outputs**: These define the data (tensors) that enter and leave the overall graph, including information on their data types and shapes.
  - **Initializers** (Weights/Parameters): These are named, constant tensor values that define the pre-trained weights and biases of the model. When an initializer has the same name as a graph input, it serves as a default value.
  - **Value Information**: Optional information regarding the types and shapes of intermediate values (tensors) produced and consumed within the graph.
- **Operator Sets** (Opsets): A model specifies the collection of operator sets (identified by a domain and version number) that define the available operators and their semantics (behavior). This ensures consistency across different runtimes.
- **Functions** (Optional): An optional list of functions local to the model, which are custom operators defined as a sub-graph of other, more primitive ONNX operators. [2, 3, 4, 5, 6]

[1] https://huggingface.co/blog/ngxson/common-ai-model-formats
[2] https://www.tutorialspoint.com/onnx/onnx-file-format.htm [3] https://www.ultralytics.com/glossary/onnx-open-neural-network-exchange [4] https://nx.docs.scailable.net/for-data-scientists/about-onnx
[5] https://mmapped.blog/posts/37-onnx-intro [6] https://github.com/onnx/onnx/blob/main/docs/IR.md

Appendix B: References

This appendix includes references to resources, standards, technologies, and models used within this guide.

CycloneDX references and resources

- OWASP Foundation
  - OWASP Dependency-Track
  - OWASP Software Component Verification Standard (SCVS)
    - SCVS BOM Maturity Model
- OWASP CycloneDX
  - CycloneDX Authoritative Guide to SBOM
  - CycloneDX Property Taxonomy
  - CycloneDX Tool Center
  - CycloneDX BOM Repository Server
- Package-URL (PURL) Specification (GitHub)

Regulatory references and standards

Regulatory references

- Ecma Technical Committee 54 - Software and System Transparency. - Standardizing core data formats, APIs, and algorithms that advance software and system transparency.
  - ECMA-424 BOM Specification - The CycloneDX specification for describing software, hardware and data components, services, dependencies, composition, attestations, vulnerabilities, licenses, formulations and more.
  - ECMA-427 PURL Specification - This standard defines the Package-URL (PURL) syntax for identifying software packages independently from their ecosystem or distribution channel
  - ECMA-428 Common Lifecycle Enumeration (CLE) specification - The CLE provides a standardized format for communicating software component lifecycle events in a machine-readable format.
- European Union's Cyber Resilience Act (EU CRA)
  - Cyber Resilience Act (CRA) - "The Final Text"
- EU AI Act (index) - The European Union's comprehensive legal framework for artificial intelligence, designed to ensure that AI systems used in the European Union are safe, ethical, and trustworthy.
  - Article 53: Obligations for Providers of General-Purpose AI Models
  - Annex XI: Technical Documentation Referred to in Article 53(1), Point (a) – Technical Documentation for Providers of General-Purpose AI Models
  - Explanatory Notice and Template for the Public Summary of Training Content for general-purpose AI models

Standards

- Linux Foundation projects
  - System Package Data Exchange™ (SPDX®)
    - SPDX License IDs
    - SPDX License List

- NIST AI Risk Management Framework
    - NIST Artificial Intelligence Risk Management Framework (AI RMF 1.0) (PDF) - A flexible guide designed to help organizations manage AI-related risks and promote trustworthy AI development.

Technology references

The following AI or ML technologies were referenced in discussion and/or examples in this guide:

- Hugging Face - an open-source platform and community for Artificial Intelligence (AI) and Machine Learning (ML).
    - Hugging Face Transformers - a library that simplifies the training and inference of models that have a transformer architecture which uses PyTorch types and implementations "under-the-covers".
- llama.cpp - an open-source inference engine, written in C++, designed for high-performance Large Language Model (LLM) execution across diverse hardware.
- PyTorch - an optimized tensor library and framework used for deep learning on GPUs and CPUs.
- TensorFlow - An end-to-end open source machine learning platform.
    - Tensorflow ModelCard Toolkit (archived) - streamlines and automates generation of Model Cards, machine learning documents that provide context and transparency into a model's development and performance.

Model references

The following ML models were referenced in discussion and/or examples in this guide:

Huggingface

Hugging Face model (repositories) typically support the .safetensors Hugging Face format; however, within the same repository, alternative formats are often found, such as PyTorch (.bin, .pt), GGUF (.gguf)

- microsoft/resnet-50 - single model.safetensors, pytorch_model.bin file.
- Qwen/Qwen-7B - multiple *.safetensors files with model.safetensors.index.json index.
    - ArXiv - STEM: Efficient Relative Capability Evaluation of LLMs through Structured Transition Samples - Analysis of Qwen3 model performance.
- Qwen/Qwen3-8B-GGUF - Contains GGUF format (i.e., .gguf files) which contain quantized versions of the Qwen3 large language model which contains both dense and mixture-of-experts (MoE) architecture models.
    - ArXiv - Qwen3 Technical Report

Kaggle

- mistral-ai/ministral-3 - multiple files that appear much like they would in a HF repo. Multiple *.safetensors files with model.safetensors.index.json index.

ONNX

ONNX models are typically single file format ending with the .onnx extension.

> ***Note**: Most ONNX models have transitioned to and are now registered in Huggingface, but are downloaded from linked GitHub repository files not within the HF repo. itself.*

- Hugging Face
    - onnx/DenseNet-121-9 - densenet-9.onnx

- GitHub
    - vision/object_detection_segmentation/tiny-yolov2/model - tinyyolov2-7.onnx

---

Benchmark (dataset) references

These are primarily references to benchmarking datasets that have been featured in examples within the guide.

- MMLU benchmark (Hugging Face - nyu-mll/glue) - MMLU consists of 15,908 multiple-choice questions, with 1,540 of them being used to select and assess optimal settings for models – temperature, batch size and learning rate. The questions span across 57 subjects, from highly complex STEM fields and international law, to nutrition and religion. It was one of the most commonly used benchmarks for comparing the capabilities of large language models.
- GLUE benchmark (zilliz.com) - The GLUE (General Language Understanding Evaluation) Benchmark is a collection of nine natural language processing (NLP) tasks designed to evaluate the performance of models on a wide range of language understanding challenges. These tasks include textual entailment, sentiment analysis, sentence similarity, and more.

Appendix A: ML-BOM mappings to the European Union's Artificial Intelligence Act (EU AI Act)

This appendix maps the EU's AI Act prose requirements and the more prescriptive Explanatory Notice and Template for the Public Summary of Training Content for general-purpose AI models to CycloneDX ML-BOM as documented in specific sections of this guide.

These mappings include:

- Article 53: Obligations for Providers of General-Purpose AI Models
- ANNEX XI: Technical Documentation Referred to in Article 53
- Annex: Template for the Public Summary of Training Content for General-Purpose AI models

> ***Disclaimer***: *Mappings are provided as a technical approach to show how the CycloneDX standard could be used to normatively record and provide information that aligns with EU AI Act requirements. Actual reporting requirements are subject to the requesting regulatory party.*

Overview of the EU AI Act

The AI Act requires model providers to report extensive model information for risk assessment and compliance. This act effectively endorses moving away from the current non-normative publication of model cards and research papers (or similar or documentation) towards normative and standardized methods such as AI/ML Bills-of-Materials (AI/ML-BOMs). Specifically, AIBOMs are recognized as a key method for creating the technical documentation required by the EU AI Act (Article 11 and Annex IV).

To fulfill the act's requirements, providers must create and maintain up-to-date technical documentation detailing the model's capabilities, limitations, and intended use.

Some of these model documentation requirements include:

- General description, architecture, number of parameters and capabilities.
- Training data provenance, methodologies and scope.
- Evaluation results and performance benchmarks.
- Known limitations and intended use cases.
- Disclosing energy consumption and other environmental impacts.

Background for the Explanatory Notice and Template for the Public Summary of Training Content for general-purpose AI models

On July 24, 2025, the European Commission released the mandatory Explanatory Notice and Template for the Public Summary of Training Content for General-Purpose AI (GPAI) models, a key compliance step under Article 53(1)(d) of the EU AI Act. This template serves as a mandatory minimum baseline for all GPAI providers, including those using open-source licenses, to publicly disclose information about their training data.

EU AI Act & Explanatory template mappings

This section provides mappings of the EU AI Act's written and templated requirements to sections of this guide that show how the CycloneDX standard could accommodate recording the corresponding information described.

Article 53: Obligations for Providers of General-Purpose AI Models

This section contains mappings to guide sections along with commentary for the EU AI Act Article 53: Obligations for Providers of General-Purpose AI Models which is part of Chapter V: General-Purpose AI Models.

Article 53 mappings

Section	Text	Guide references & commentary
1.	Providers of general-purpose AI models shall:	*N/A*
1.(a)	draw up and keep up-to-date the technical documentation of the model, including its training and testing process and the results of its evaluation, which shall contain, at a minimum, the information set out in Annex XI for the purpose of providing it, upon request, to the AI Office and the national competent authorities;	See ANNEX XI: mappings
1.(b)	draw up, keep up-to-date and make available information and documentation to providers of AI systems who intend to integrate the general-purpose AI model into their AI systems. Without prejudice to the need to observe and protect intellectual property rights and confidential business information or trade secrets in accordance with Union and national law, the information and documentation shall:	This effectively describes the AI/ML BOM document in its entirety.
1.(b).(i)	enable providers of AI systems to have a good understanding of the capabilities and limitations of the general-purpose AI model and to comply with their obligations pursuant to this Regulation; and	Model design considerations
1.(b).(ii)	contain, at a minimum, the elements set out in Annex XII;	Annex XII: "Technical Documentation for Providers of General-Purpose AI Models to Downstream Providers that Integrate the Model into Their AI System" **Note**: CycloneDX can fully describe an AI/ML model that is part of, or used by, an application or service via contextual or referential inclusion as components in a Software Bill-of-Materials (SBOM) or Software-as-a-Service Bill-of-Materials (SaaSBOM).

Section	Text	Guide references & commentary
1.(c)	put in place a policy to comply with Union law on copyright and related rights, and in particular to identify and comply with, including through state-of-the-art technologies, a reservation of rights expressed pursuant to Article 4(3) of Directive (EU) 2019/790;	**Intention**: *Article 4 in Directive 2019/790 (CDSMD), the European Union legislator intended to both encourage innovation and to provide more legal certainty for text and data mining (TDM) activities.* • See commentary: Oxford: Journal of Intellectual Property Law & Practice - "The text and data mining opt-out in Article 4(3) CDSMD: Adequate veto right for rightholders or a suffocating blanket for European artificial intelligence innovations?" CycloneDX enables various methods of conveying non-normative and legal information. Primarily, this is accomplished via externalReferences, component properties, as well as through explicit licenses objects and copyright fields.
1.(d)	draw up and make publicly available a sufficiently detailed summary about the content used for training of the general-purpose AI model, according to a template provided by the AI Office.	The CycloneDX model card's parameter object allows for a description of the model's Training Approach. **Note**: CycloneDX, as a Bills-of-Materials standard, accounts for each dataset as its own, fully described, component. See Declaring Datasets.
2.	The obligations set out in paragraph 1, points (a) and (b), shall not apply to providers of AI models that are released under a free and open-source licence that allows for the access, usage, modification, and distribution of the model, and whose parameters, including the weights, the information on the model architecture, and the information on model usage, are made publicly available. This exception shall not apply to general-purpose AI models with systemic risks.	*N/A*
3.	Providers of general-purpose AI models shall cooperate as necessary with the Commission and the national competent authorities in the exercise of their competences and powers pursuant to this Regulation.	*N/A*

Section	Text	Guide references & commentary
4.	Providers of general-purpose AI models may rely on codes of practice within the meaning of Article 56 to demonstrate compliance with the obligations set out in paragraph 1 of this Article, until a harmonised standard is published. Compliance with European harmonised standards grants providers the presumption of conformity to the extent that those standards cover those obligations. Providers of general-purpose AI models who do not adhere to an approved code of practice or do not comply with a European harmonised standard shall demonstrate alternative adequate means of compliance for assessment by the Commission.	Article 56 references new "Codes of Practice" that provide: • a means to ensure that the information, referred to in Article 53, is kept up to date. • an adequate level of detail for the summary about the content used for training; • identification of the type and nature of the systemic risks at Union level, including their sources, where appropriate; • measures, procedures and modalities for the assessment and management of the systemic risks at Union level, including the documentation thereof, which shall be proportionate to the risks. Future guide revisions will provide updates as these codes are developed to facilitate CycloneDX compliance.
5.	For the purpose of facilitating compliance with Annex XI, in particular points 2 (d) and (e) thereof, the Commission is empowered to adopt delegated acts in accordance with Article 97 to detail measurement and calculation methodologies with a view to allowing for comparable and verifiable documentation.	In short, Article 97 provides for *"The power to adopt delegated acts is conferred on the Commission subject to the conditions laid down in this Article.* which would *"enter into force upon only if no objection has been expressed by either the European Parliament or the Council within a period of three months of notification*. As any additional "delegated acts" are developed, future revisions of this guide will provide updates to facilitate compliance using CycloneDX.
6.	The Commission is empowered to adopt delegated acts in accordance with Article 97(2) to amend Annexes XI and XII in light of evolving technological developments.	*See commentary provided to paragraph 5.*

Section	Text	Guide references & commentary
7.	Any information or documentation obtained pursuant to this Article, including trade secrets, shall be treated in accordance with the confidentiality obligations set out in Article 78.	In short, Article 78 provides assurances to providers that their *"the intellectual property rights and confidential business information or trade secrets of a natural or legal person, including source code"* will remain confidential and protected by law when handled by regulators and officials. The CycloneDX Bills-of-Materials (BOMs) format can be used to convey any information a provider chooses to encode subject to their legal discretion.

ANNEX XI: Technical Documentation Referred to in Article 53(1), Point (a) – Technical Documentation for Providers of General-Purpose AI Models

This section contains mappings for ANNEX XI: Technical Documentation Referred to in Article 53.

Annex XI mappings

Section	Section text	Guide references	Relevant schema (v1.7)
1	Information to be provided by all providers of general-purpose AI models The technical documentation referred to in Article 53 (1), point (a) shall contain at least the following information as appropriate to the size and risk profile of the model:	*See details in subsections below*	N/A
1.1	A general description of the general-purpose AI model including:	CycloneDX describes models as components: • Declaring ML models	• metadata.component. ■ type: "machine-learning-model" ■ name ■ version ■ description ■ supplier ■ manufacturer ■ Component publisher
1.1.(a)	the tasks that the model is intended to perform and the type and nature of AI systems in which it can be integrated;	Use cases and users: • Considerations: Users & use cases	• metadata.component.modelCard. ■ considerations. ■ users ■ useCases

Section	Section text	Guide references	Relevant schema (v1.7)
1.1.(b)	the acceptable use policies applicable;	Use policies: • Providing a model's usage policy *- See example for the Qwen model.*	Usage policies can be provided as a CycloneDX external reference. • metadata.component.externalReferences **Note**: multiple references to published usage policies can be provided.
1.1.(c)	the date of release and methods of distribution;	Release information: • Providing model release notes	Release dates and methods are part of CycloneDX are provided using the releaseNotes fields in the model's component: • metadata.component.releaseNotes. ■ type ■ description ■ timestamp ■ notes ■ etc. **Note:** *Components support multiple releases notes for the associated model/ version.*
1.1.(d)	the architecture and number of parameters;	Model architecture: • Architecture family • Model architecture	• metadata.component.modelCard. • modelParameters. ■ architectureFamily ■ modelArchitecture
1.1.(e)	the modality (e.g. text, image) and format of inputs and outputs;	Modality/modalities: • Declaring a model's modalities Inputs & Outputs • Inputs & Outputs	Inputs & Outputs: • modelCard.modelParameters. ■ inputs ■ outputs Modality/modalities: • metadata.component.properties
1.1.(f)	the licence.	Component license: • Describing models as components ■ *See Example: Declaring an ML model in an ML-BOM which uses the CycloneDX license object.*	CycloneDX provides multiple, robust options for recording license information: • metadata.licenses
1.2	A detailed description of the elements of the model referred to in point 1, and relevant information of the process for the development, including the following elements:	*See details in subsections below*	N/A

Section	Section text	Guide references	Relevant schema (v1.7)
1.2.(a)	the technical means (e.g. instructions of use, infrastructure, tools) required for the general-purpose AI model to be integrated in AI systems;	Detailing inference workflows, tasks, steps and resources to be used in testing and production: • Including manufacturing information for the ML model • Declaring the runtime topology	Detailed inference and testing workflows: • formulation. ■ workflows ■ tasks ■ steps ■ runtimeTopology **Note**: The CycloneDX formulation object can be used to convey workflows for such things as inference in the context of various target frameworks or their runtime topologies.
1.2.(b)	the design specifications of the model and training process, including training methodologies and techniques, the key design choices including the rationale and assumptions made; what the model is designed to optimise for and the relevance of the different parameters, as applicable;	Considerations when designing the model: • Technical limitations • Performance tradeoffs • Ethical considerations Describing design, data preparation and training workflows along with detailed tasks, steps and resources: • Including manufacturing information for the ML model • Declaring hardware and software training components ■ Providing training workflow details ■ Declaring the runtime topology	Considerations: • metadata.component.modelCard. ■ considerations. ■ technicalLimitations ■ performanceTradeoffs ■ ethicalConsiderations Detailed workflows: • formulation. ■ workflows ■ tasks ■ steps ■ runtimeTopology **Note**: CycloneDX v2.0 will have extensible workflow taskTypes that will include an AI/ML taxonomy with values for such things as training or fine-tuning.

Section	Section text	Guide references	Relevant schema (v1.7)
1.2.(c)	information on the data used for training, testing and validation, where applicable, including the type and provenance of data and curation methodologies (e.g. cleaning, filtering, etc.), the number of data points, their scope and main characteristics; how the data was obtained and selected as well as all other measures to detect the unsuitability of data sources and methods to detect identifiable biases, where applicable;	Data or dataset declaration, provenance and pedigree: • Datasets ■ Declaring datasets ■ datasets as component references ■ datasets as in-line declarations	Detailed data preparation workflows: • formulation. ■ workflows ■ tasks ■ steps ■ runtimeTopology
1.2.(d)	the computational resources used to train the model (e.g. number of floating point operations), training time, and other relevant details related to the training;	Detailing training workflows, tasks, steps and resources: • Including manufacturing information for the ML model ■ Declaring hardware and software training components ■ Providing training workflow details ■ Declaring the runtime topology	Detailed training workflows: • formulation. ■ workflows ■ tasks ■ steps ■ runtimeTopology **Note**: CycloneDX v2.0 will have extensible workflow taskTypes that will include an AI/ML taxonomy with values for such things as training or fine-tuning.
1.2.(e)	known or estimated energy consumption of the model. With regard to point (e), where the energy consumption of the model is unknown, the energy consumption may be based on information about computational resources used.	Per-activity energy consumptions, energy provider information, CO2 costs and CO2 cost offsets: • Energy Consumptions	• Environmental Considerations ■ Energy Consumptions ■ activity ■ energyProviders ■ activityEnergyCost ■ co2CostEquivalent ■ co2CostOffset **Note**: *Energy consumptions can be reported on a per-activity basis (e.g., data-collection, t raining, fine-tuning, etc.) and can correspond to declared workflows.*
2	Additional information to be provided by providers of general-purpose AI models with systemic risk	*See details in subsections below*	N/A

Section	Section text	Guide references	Relevant schema (v1.7)
2.1	A detailed description of the evaluation strategies, including evaluation results, on the basis of available public evaluation protocols and tools or otherwise of other evaluation methodologies. Evaluation strategies shall include evaluation criteria, metrics and the methodology on the identification of limitations.	Describing and recording results for performance (evaluation) tests: • Model quantitative analysis ■ Performance Metrics ■ Graphics	• metadata.component.modelCard. ■ considerations. ■ quantitativeAnalysis. ■ performanceMetrics ■ graphics **Note:** Evaluation processes can be encoded as formulation workflows, and their compositional tasks, steps, tools, inputs, outputs and more.
2.2	Where applicable, a detailed description of the measures put in place for the purpose of conducting internal and/or external adversarial testing (e.g. red teaming), model adaptations, including alignment and fine-tuning.	Additionally, "Ethical considerations" and "Fairness assessments" can be documented as shown in these sections: • Fairness assessments	• metadata.component.modelCard. considerations. • fairnessAssessments
2.3	Where applicable, a detailed description of the system architecture explaining how software components build or feed into each other and integrate into the overall processing.	The composition of model components, including data: • Declaring ML Models • Describing models as components • Model repositories as components • Describing a model repository as a CycloneDX assembly	Component and service relationships: • compositions. (nested relationships) • assemblies • dependencies (required, non-transitive relationships) Hierarchical (nested) relationships (for assemblies): • metadata.component.components • components.components • services.services Direct dependencies: • dependencies Process and data flows: • formulation. ■ workflows ■ tasks ■ runtimeTopology **Note**: *When models are incorporated into hardware and software systems, CycloneDX supports declaring full dependency relationships as well as detailing service and data processing workflows.*

Annex: Template for the Public Summary of Training Content for General-Purpose AI models required by Article 53

This section provides mappings for the *"Explanatory Notice"* and *"Template for the Public Summary of Training Content"* which seek to address relevant legal text from Article 53(1)(d) of the AI Act:

Providers of general-purpose AI models shall […] draw up and make publicly available a sufficiently detailed summary about the content used for training of the general-purpose AI model, according to a template provided by the AI Office.

As well as Recital 107 of the AI Act:

In order to increase transparency on the data that is used in the pre-training and training of general-purpose AI models, including text and data protected by copyright law, it is adequate that providers of such models draw up and make publicly available a sufficiently detailed summary of the content used for training the general-purpose AI model.

Template mapping notes

Subsections under Section 2, *"Lists of data sources"*, require similar information about data or datasets and their collection processes using different techniques and from different sources. Therefore, much of the *"Guide references"* and *"CycloneDX Commentary"* text will be similar across the following subsections:

- 2.1 Publicly available datasets
- 2.2 Private non-publicly available datasets obtained from third parties
- 2.3 Data crawled and scraped from online sources
- 2.4 User data
- 2.5 Synthetic data
- 2.6 Other data sources

Template mappings

Section	Text	Guide references	CycloneDX Commentary
1.	General information	See Annex XI mappings, Section 1.1 • Declaring ML models	The majority of this information would be provided within the CycloneDX component.metadata for the model.
1.1	Provider identification	See Annex XI mappings, Section 1.1 • Declaring ML models	Manufacturer, supplier and publisher information can be provided within the model's metadata: • manufacturer - *The organization that built or created the model.* • supplier - *The organization that supplied the model for use* • publisher - *The organization that published the model*
1.1.(i)	Provider name and contact details	See template mappings, Section 1.1 (above)	Both the manufacturer and supplier information includes: • name, address, url and multiple (i.e., an array of), detailed contact information which accounts for multiple points-of-contact. Component publisher information supports a textual description.
1.1.(ii)	Authorised representative name and contact details	See template mappings, Section 1.1.(i) (above)	Each contact information includes: • name, email address and phone

Section	Text	Guide references	CycloneDX Commentary
1.2	Model identification	A discussion of model identifiers with examples: • Model identifiers	The model component information includes support for the Package-URL (PURL) Specification specification which provides syntax for identifying a model from various source repositories: • metadata.component. ■ purl **Note**: *In addition, CycloneDX provides the means to provide identifiers from registered proprietary or other publication sources via the CycloneDX property taxonomy.*
1.2.(i)	Versioned model name(s)	A model's name, identifiers and version are considered unique attributes of the model component. These are provided in the metadata.component field as shown here: • Describing models as components • Model identifiers If a model is derived from another model, that relationship would be described by pedigree: • Declaring a model's pedigree	Model name, version and identifiers: • metadata.component Model pedigree: • component.pedigree **Note**: *An AI/ML Bill-of-Materials is intended to represent a single identifiable model. Each separate version published or supplied from a different source, should be represented by its own unique AI/ML-BOM which can capture its pedigree, including any changes from the original model, by reference to its AI/ML BOM via pedigree fields.*

1.2.(ii)	Model dependencies	Dependencies of a model needs to consider both the datasets used for training and/or finetuning as well as listing dependencies on tokenizers, templates and any configurations. Accounting for these resources are shown in the following sections, respectively:
		• Model repositories as components
		• Declaring datasets
		  ■ Datasets as data component references
			Model component and service compositions (e.g., datasets, tensor data, tokenizers, configurations, etc.):
			• compositions. (nested relationships)
			  • assemblies
			  • dependencies (required, non-transitive relationships)
			
			Hierarchical (nested) relationships (for assemblies):
			• metadata.component.components
			• components.components
			• services.services
			
			Direct dependencies:
			• dependencies
			
			Explicit declaration of model datasets used (using CycloneDX data component references):
			• modelCard.modelParameters.datasets
			
			Process and data flows:
			• formulation.
			  ■ workflows
			  ■ tasks
			  ■ runtimeTopology

Section	Text	Guide references	CycloneDX Commentary
1.2.(iii)	Date of placement of the model on the Union market:	This information would be provided in the model's release notes: • Providing model release notes	Release notes: • component.releaseNotes
1.3.	Modalities, overall training data size and other characteristic *(general information about the overall training data after pre-processing and before the training of the model)*	*See details in subsections below*	N/A
1.3.(i)	Modality *(e.g., text, image, audio, video, other)*	Model modalities: • Declaring a model's modalities **Note**: *Multi-model models should include modality information for each sub-model.*	Modalities: • metadata.component.properties **Note**: *Utilizes property values defined in the the CycloneDX Property Taxonomy for AI/ML*
1.3.(ii)	Training data size	The CycloneDX component can be used to describe a training dataset with any level of detail required. In general, this section describes the general method on how to declare public and private datasets: • Declaring datasets Additionally, other types of information about each component dataset can be provided via various fields such as: • *pedigree*` • *external references* to documentation • *properties* for customized for tagging information to domain-specific requirements such as data sizes.	Dataset component(s): • component ■ pedigree ■ externalReferences ■ properties **Note**: *Ideally, each dataset would have its own independent Bill-of-Materials that fully described the details of its design, dependencies (i.e., data sources) and manufacturing which could be referenced by the AI/ML BOM.*

Section	Text	Guide references	CycloneDX Commentary
1.3.(iii)	Types of content	A discrete description of the types of content used to train a model would be provided as CycloneDX data components. • Declaring datasets Additional content information can be provided via external documentation and referenced in the model's component declaration. • Providing links to papers & articles	Dataset components, their descriptions and external references to documentation: • component. ■ type: "data" ■ description ■ etc. Model component's external references: • metadata.component.externalReferences
2	List of data sources *(information about specific sources of data used to train the general-purpose AI model)*	*See details in subsections below*	N/A
2.1	Publicly available datasets	Each *public* dataset used to train a model would be provided as a CycloneDX data component. • Declaring datasets ■ Datasets as in-line information ■ Datasets as data component references	Dataset component(s): • component ■ [type]: "data" ■ name ■ description ■ pedigree ■ externalReferences ■ properties ■ etc. **Note**: *Ideally, each public dataset would have its own independent Bill-of-Materials that fully described the details of its design, dependencies (i.e., data sources) and manufacturing which could be referenced by the AI/ML BOM.*
2.2	Private non-publicly available datasets obtained from third parties	Private dataset information would be provided similarly to public datasets. See template mappings, Section 2.1 *"Publicly available data"* (above)	*See referenced section.*

Section	Text	Guide references	CycloneDX Commentary
2.2.1	Datasets commerciall y licensed by rightsholders or their representativ es	Commercial dataset information would be provided similarly to public datasets. See template mappings, Section 2.1 *"Publicly available data"* (above)	*See referenced section.*
2.2.1.(i)	concluded transactional commercial licensing agreement (modalities covered by license)	License information would be provided in the CycloneDX data component: • Describing models as components ■ *Example: Declaring an ML model in an ML-BOM* which uses the CycloneDX license object.	CycloneDX provides multiple, robust options for recording license information: • metadata.licenses **Note**: *modality-specific licensing may have considerations in future CycloneDX versions.*
2.2.2	Private datasets obtained from other third parties	Third-party, private dataset information would be provided similarly to public datasets. See template mappings, Section 2.1 *"Publicly available data"* (above)	*See referenced section.*
2.2.2.(i)	Specify the modality(ies) of the content covered by the datasets concerned.	Model data component modalities are declared in the same way as for the model component itself: • Declaring a model's modalities	Data component modalities as properties: • component.properties **Note**: *Utilizes property values defined in the the CycloneDX Property Taxonomy for AI/ML*
2.2.2. (ii)	If publicly known, list private datasets obtained from other third parties	Publicly known, third-party, private dataset information would be provided similarly to public datasets. See template mappings, Section 2.1 *"Publicly available data"* (above)	*See referenced section.*
2.2.2. (iii)	General description of non-publicly known private datasets obtained from third parties	Non-publicly known, third-party, private dataset information would be provided similarly to public datasets. See template mappings, Section 2.1 *"Publicly available data"* (above)	*See referenced section.*

Section	Text	Guide references	CycloneDX Commentary
2.2.2. (iv)	Additional comments *(optional)* *e.g., the period of data collection, size of the datasets and further details*	CycloneDX Bills-of-Materials (e.g., an AI/ML BOM) supports annotations that allow for comments (made by people, organizations, or tools) about any object with a bom-ref such as components, services or the BOM itself.	Comments using annotations: • annotations ■ subjects - *list of references (e.g., components, services, etc.) the annotation applies to.* ■ annotator - *The organization, person, component, or service which created the textual content of the annotation.* ■ timestamp ■ text - *The textual content of the annotation.* ■ signature *(optional) - digital signature of the signer.* **Note**: *Each annotation can optionally have its own unique bom-ref which allows reference from other annotations.*
2.3	Data crawled and scraped from online sources *(excluding publicly available datasets already compiled by third parties and made available on platforms such as common crawl that are covered under Section 2.1)* *The following subsections only apply if "crawlers were used for data collection".*	*While this guide does not provide specific examples for data crawling, the training workflow example can serve as a template for extrapolating how to document the methods and data sources using CycloneDX workflows.* See Annex XI mappings, Section 1.2.(b) (above)	Any data collection processes would be described using CycloneDX workflows: • formulation. ■ workflows ■ tasks ■ runtimeTopology **Note**: *Best practices would have the data collection processes, for the resultant subject named dataset, captured in a Manufacturing Bill-of-Materials (or MBOM) which could be referenced by an AI/ML BOM for a model that used that dataset for training or finetuning.*

Section	Text	Guide references	CycloneDX Commentary
2.3.(i)	specify crawler name(s)/ identifier(s)	The *data crawler* would be declared as a CycloneDX component with its name and identifiers provided as described for a model component. See applicable references in the following template mappings table sections (above): • Section 1.2, "Model identification" • Section 1.2.(i), "Versioned model name(s)"	*See referenced sections.*
2.3.(ii)	Purposes of the crawler(s)	The crawler software would be declared as a CycloneDX component (or service) which can include its description, documentation via externalReferences, properties and annotations. See template mappings, Section 2.1, "Publicly available datasets" (above) for how to include component information and template mappings, Section 2.2.2.(iv), "Additional comments" for annotations.	*See referenced sections.*
2.3.(iii)	General description of crawler behaviour	See referenced methods for annotations in template mappings, Section 2.3.(i) (above)	*See referenced section.*
2.3.(iv)	Period of data collection	See referenced methods in template mappings, Section 2.2.2.(iv) (above)	*See referenced section.*
2.3.(v)	Comprehensive description of the type of content and online sources crawled	Each content source the crawler software targeted would be declared as a CycloneDX component with the ability to describe the content names, identifiers and location. See template mappings, Section 2.1, "Publicly available datasets" (above) for how to include component information.	*See referenced section.*

Section	Text	Guide references	CycloneDX Commentary
2.3.(vi)	Type of modality covered	Each content source can include modality properties as referenced in template mappings, Section 2.2.2.(i) (above).	*See referenced section.*
2.3.(vii)	Summary of the most relevant domain names crawled	The BOM for the software crawler should have a detailed listing of all crawled sources represented as CycloneDX components (or services). This comprehensive information could be provided using the crawler's BOM itself or the BOM used to produce a summary view.	N/A
2.3. (viii)	Additional comments *(optional)* *e.g., domain names, URLs and the sources of individual works*	See template mappings, Section 2.2.2.(iv) (above)	*See referenced section.* ***Note**: Additionally, CycloneDX workflows that describe data collection processes can contain optional startTime and endTime timestamps for the overall process and each of its composing tasks.*
2.4	User data *(information about user data collected by all services and products of the provider, including through mail services, social media platforms, content platforms)* *The following subsections only apply if user information sources were used.*	*While this guide does not provide specific examples for documenting user data collection, the training workflow example can serve as a template for extrapolating how to achieve this using CycloneDX.* See Annex XI mappings, Section 1.2.(b) (above)	Any data collection processes would be described using CycloneDX workflows: • formulation. ■ workflows ■ tasks ■ runtimeTopology **Note**: *Best practices would have the data collection processes, for the resultant subject named dataset, captured in a Manufacturing Bill-of-Materials (or MBOM) which could be referenced by an AI/ML BOM for a model that used that dataset for training or finetuning.*

Section	Text	Guide references	CycloneDX Commentary
2.4.(i)	provide a general description of the provider's services or products that were used to collect the user data	Data collection processes can be encoded using CycloneDX formulation workflows which can include full descriptions of the provider's services or products (i.e., using CycloneDX service and component definitions) used and formulations and reference them discretely, as resources, in the workflow's tasks where they were used.	Any services or software used in data collection processes would be described in CycloneDX formulations: • formulation. ■ components ■ services ■ workflows ■ tasks ■ resourceReferences
2.4.(ii)	Additional comments *(optional)*	See referenced methods for annotations in template mappings, Section 2.3.(i) (above)	*See referenced section.*
2.5	Synthetic data *The following subsections only apply if synthetic information sources were used.*	*While this guide does not provide specific examples for capturing synthetic data generation processes, the training workflow example can serve as a template for extrapolating how to document the methods and data sources using CycloneDX workflows.* See Annex XI mappings, Section 1.2.(b) (above)	*See referenced section.*
2.5.(i)	modality of the synthetic data	See referenced methods for providing modalities in template mappings, Section 2.2.2.(i) (above)	*See referenced section.*
2.5.(ii)	specify the general-purpose AI model(s) used to generate the synthetic data if available on the market	Any model used to generate synthetic data can be described using its own CycloneDX model component which could be listed as a resource within the workflow that describes the synthetic data generation process. See section Annex XI mappings, Section 1.1 (above)	*See referenced section.*

Section	Text	Guide references	CycloneDX Commentary
2.5.(iii)	Information about other AI models, including provider's own AI model(s) not available on the market, used to generate synthetic data to train the model	Any models used to generate *Synthetic data* would themselves be declared as a CycloneDX components. See Annex XI mappings, Section 1.1 (above)	*See referenced section.*
2.5.(iv)	Additional comments *(optional)*	See Section 2.2.2.(iv) (above)	*See referenced section.*
2.6	Other sources of data *The following subsections only apply if other information sources were used.*	*While this guide does not provide specific examples for capturing data from other sources, the training workflow example can serve as a template for extrapolating how to document the methods and data sources using CycloneDX workflows.* See "Annex XI mappings, Section 1.2.(b)" (above)	*See referenced section.*
2.6.(i)	provide a narrative description of these data sources and the data	See referenced methods for annotations in Section 2.3. (i) (above)	*See referenced section.*
2.6.(ii)	Additional comments *(optional)*	See referenced methods for annotations in Section 2.3. (i) (above)	*See referenced section.*
3	Data processing aspects The following subsections only apply if synthetic information sources were used.	*See details in subsections below*	N/A

Section	Text	Guide references	CycloneDX Commentary
3.1	Respect of reservation of rights from text and data mining exception or limitation *(measures implemented by the provider to identify and comply with the reservation of rights from the text and data mining (TDM) exception or limitation expressed pursuant to Article 4(3))*	*While this guide does not provide specific examples on providing measure implemented to comply with the reservation of rights from the text and data mining (TDM) exception and similar requirements, descriptive steps can be provided for each dataset's data acquisition process to show implementation of these measures.*	Detailed data acquisition steps with implementation descriptions: • formulation. ■ workflows ■ tasks ■ steps Specifically, each task and step of a workflow has a description field as well as properties where domain-specific named properties and values can be recorded. Additionally, components used to acquire data can be annotated to include information about their configurations and policies with perhaps links to these as their own component artifacts. See template mappings, Section 2.2.2.(iv) (above)
3.1.(i)	Additional comments *(optional)*	See template mappings, Section 2.2.2.(iv) (above)	*See referenced section.*

Section	Text	Guide references	CycloneDX Commentary
3.2	Removal of illegal content *measures taken to avoid or remove illegal content under Union law from the training data (such as blacklists, keywords, and model-based classifiers), without requiring disclosure of specific details about the provider's internal business practices or trade secrets*	*While this guide does not provide specific examples for content removal or filtering, the training workflow example can serve as a template for extrapolating how to document the methods and data sources using CycloneDX workflows.* See references in table "Annex XI mappings, Section 1.2.(c)" (above)	Detailed data preparation workflows: • formulation. ■ workflows ■ tasks ■ steps
3.3	Other information *(optional)* *Other relevant information about data processing*	CycloneDX allows organizations to document descriptive details for each dataset's processing and preparation workflow. See references in "Annex XI mappings, Section 1.2.(c)" (above)	Detailed data processing workflows: • formulation. ■ workflows ■ tasks ■ steps Specifically, each task and step of a workflow has a description field as well as properties where domain-specific, named properties and values can be recorded.

Appendix C: References

This appendix includes a complete AI/ML BOM example that combines most of the isolated examples for the Qwen model shown throughout this guide.

Example: Qwen-7B AI/ML BOM

> ***Note**: For brevity, the formulation entry for the model's training only describes the top-level workflow topology (i.e., the run-time "stack"), but none of the tasks or steps that could be detailed.*


```
{
  "$schema": "http://cyclonedx.org/schema/bom-1.7.schema.json",
  "bomFormat": "CycloneDX",
  "specVersion": "1.7",
  "serialNumber": "urn:uuid:ec45525e-516c-4405-9de3-4fbdaef7f09a",
  "version": 1,
  "metadata": {
    "component": {
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9",
      "purl": "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9c57b252f3149c1408daf4d649ec8b6c85",
      "group": "Qwen",
      "manufacturer": {
        "name": "Alibaba Cloud",
        "url": [ "https://www.alibabacloud.com" ]
      },
      "supplier": {
        "name": "Hugging Face",
        "url": [ "https://huggingface.co" ]
      },
      "name": "Qwen/Qwen-7B",
      "version": "ef3c5c9c57b252f3149c1408daf4d649ec8b6c85",
      "description": "Qwen-7B is a Transformer-based large language model, which is pretrained on a
large volume of data, including web texts, books, codes, etc.",
      "licenses": [
        {
          "license": {
            "name": "Tongyi Qianwen LICENSE AGREEMENT",
            "text": {
              "content":
"By clicking to agree or by using or distributing any portion or element of the Tongyi Qianwen
Materials, ..."
            }
          }
        }
      ],
      "releaseNotes": {
          "type": "major",
          "title": "Qwen 7B initial release",
          "timestamp": "2023-08-03T15:30:00Z",
          "notes": [
            {
              "locale": "en-US",
              "text": "United States (US), English release date."
            },
            // ...
          ]
        }
      },
      "externalReferences": [
        {
```

```
          "type": "vcs",
          "url": "https://huggingface.co/"
        },
        {
          "type": "model-card",
          "url": "https://huggingface.co/Qwen/Qwen-7B"
        },
        {
          "type": "documentation",
          "url": "https://arxiv.org/abs/2309.16609",
          "comment": "Qwen Technical Report"
        }
      ],
      "properties": [
        {
          "name": "acme:research:model:llm:id",
          "value": "MODEL-ID-12345-INTERNAL"
        },
        {
          "name": "cdx:ai-ml:model:template:chat",
          "value": "ChatML"
        }
      ],
      "tags": [
        "pytorch",
        "transformers",
        "text-generation"
      ],
      "pedigree": {
        "ancestors": [
          {
            "type": "machine-learning-model",
            "name": "meta-llama/Llama-3.2-3B-Instruct",
            "publisher": "Meta",
            "purl": "pkg:huggingface/meta-llama/Llama-3.2-3B-Instruct",
            "description": "The original base model from Meta Llama used for fine-tuning."
          }
        ]
      },
      "components": [
        {
          "type": "file",
          "name": "config.json",
          "description": "Model configuration file using the 'QWenLMHeadModel' model class in Hugging
Face Transformers",
          "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@e7a368b#config.json",
          "purl": "pkg:huggingface/Qwen/Qwen-7B@e7a368b0774370edec29674e7c51f52fc7663f59#config.json"
        },
        {
          "type": "file",
          "name": "configuration_qwen.py",
          "description": "Python 'QWenConfig' class implementation for the Qwen-7B model using Hugging
Face Transformers",
          "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@a6ca629#configuration_qwen.py",
          "purl": "pkg:huggingface/Qwen/
Qwen-7B@a6ca629d063f56f34d184852301e8852a7afbd58#configuration_qwen.py"
        },
        {
          "type": "library",
          "name": "tokenization_qwen.py",
          "description": "Python tokenization classes for QWen (QWenTokenizer)",
          "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@e7a368b#tokenization_qwen.py",
          "purl": "pkg:huggingface/Qwen/
Qwen-7B@e7a368b0774370edec29674e7c51f52fc7663f59#tokenization_qwen.py",
```

```
          "properties": [
            {
              "name": "cdx:ai-ml:model:tokenizer",
              "value": "QWenTokenizer"
            }
          ]
        },
        {
          "type": "data",
          "name": "model-00001-of-00008.safetensors",
          "description": "Model tensor data (01 of 08)",
          "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@abcb6d6#model-00001-of-00008.safetensors",
          "purl": "pkg:huggingface/Qwen/Qwen-7B@abcb6d6d8ec63ce606f816e2d08072da6309f965#model-00001-
of-00008.safetensors",
          "data": [
            {
              "type": "dataset",
              "contents": {
                "url": "https://huggingface.co/Qwen/Qwen-7B/blob/main/model-00001-of-00008.safetensors"
              }
            }
          ]
        },
        {
          "type": "data",
          "name": "model-00002-of-00008.safetensors",
          "description": "Model tensor data (02 of 08)",
          "bom-ref": "pkg:huggingface/Qwen/Qwen-7B@abcb6#model-00002-of-00008.safetensors",
          "purl": "pkg:huggingface/Qwen/Qwen-7B@abcb6d6d8ec63ce606f816e2d08072da6309f965#model-00002-
of-00008.safetensors",
          "data": [
            {
              "type": "dataset",
              "contents": {
                "url": "https://huggingface.co/Qwen/Qwen-7B/blob/main/model-00002-of-00008.safetensors"
              }
            }
          ]
        }
      ],
      "modelCard": {
        "modelParameters": {
          "task": "text-generation",
          "architectureFamily": "transformer",
          "modelArchitecture": "QWenLMHeadModel",
          "approach": {
            "type": "supervised"
          },
          "datasets": [
            {
              "type": "dataset",
              "name": "UltraFeedback dataset",
              "classification": "public",
              "contents": {
                "url": "https://huggingface.co/datasets/openbmb/UltraFeedback"
              }
            },
            {
              "type": "dataset",
              "name": "ACME Midwest health data",
              "classification": "private",
              "contents": {
                "url": "https://acme.ai/adatasets/health/patient?region=midwest"
              }
```

```
            }
          ],
          "inputs": [
            {
              "format": "string"
            }
          ],
          "outputs": [
            {
              "format": "string"
            }
          ]
        },
        "quantitativeAnalysis": {
          "performanceMetrics": [
            {
              "type": "MMLU (5-shot)",
              "value": "58.2",
              "confidenceInterval": {
                "lowerBound": "94.28",
                "upperBound": "95.72"
              }
            },
            {
              "type": "GLUE (F1 Score)",
              "value": "0.87",
              "slice": "cola"
            }
          ],
          "graphics": {
            "description": "benchmark_score",
            "collection": [
              {
                "name": "Qwen2 Performance Benchmarks (spider diagram)",
                "image": {
                  "contentType": "image/jpeg",
                  "encoding": "base64",
                  "content": "/9j/4AAQSkZJRgABAQAAAQABAAD/2wBDAA"
                }
              }
            ]
          }
        },
        "considerations": {
          "users": [
            "Software developer",
            "Multilingual Content Creator",
            "Customer Support Systems Architect",
            "Academic Researcher / Student",
            "Edge Device Engineer",
            "Enterprise Security Analyst",
            "Local AI Enthusiast / Privacy-First User"
          ],
          "useCases": [
            "Utilizing the Qwen \"instruct\" variants within an IDE for real-time code completion, bug fixing, and unit test generation, benefiting from its \"Agentic\" capabilities for repository-scale understanding.",
            "Translating business, education, or other content or informational materials to other languages and dialects while maintaining the original tone and cultural nuances.",
            "Deploying low-latency chatbots for high-volume inquiries where the 7B model acts as a \"triage\" agent, answering common questions and only escalating complex logic to other support mechanisms.",
            "Summarizing long-form research papers and generating initial drafts for school projects, utilizing the model's 128K context window to ingest entire PDFs at once.",
            "Implementing the model on specialized hardware for real-time visual perception and \"Thinki
```

```
ng Mode\" reasoning to help an intelligent device navigate and interact with its environment based on natural language commands",
            "Running a self-hosted instance to analyze internal security logs for anomalies, ensuring that sensitive infrastructure data never leaves the organization's firewall.",
            "Running a personal assistant locally on a laptop to answer questions or process private information such as emails or calendars without sending data to an external server."
          ],
          "technicalLimitations": [
            "Greedy Decoding Degradation. The model is optimized for sampling-based generation. Using greedy decoding (temperature=0) can lead to performance degradation, repetitive loops, and \"stuck\" reasoning steps, particularly in the new Thinking Mode",
            "Native Context Window Boundaries. While the model supports up to 131,072 tokens using YaRN scaling, its native pre-training context is limited to 32,768 tokens. Performance may degrade on very long sequences if proper scaling factors (like RoPE or YaRN) are not manually configured for local deployments.",
            "Synthetic Data \"Sanding\" Effects. Research indicates that Qwen, like many models trained on massive synthetic datasets, can suffer from \"model
collapse\" where rare edge cases or minority user behaviors are underrepresented, potentially leading to errors in complex, real-world production environments.",
            "Thinking Mode History Overhead. In multi-turn conversations, including the model's internal \"thinking\" steps in the chat history can confuse the model and consume unnecessary tokens. Best practices require developers to filter out \"thinking\" content from the history to maintain coherence."
          ],
          "performanceTradeoffs": [
            "Intelligence Plateau in Domain-Specific Tasks. Research indicates that for specialized fields like legal text analysis, performance often flattens beyond the 7B parameter mark. While efficient, the 7B model may not offer the incremental reasoning gains found in the 32B or 235B models for complex, high-stakes domain reasoning.",
            "Enhanced Quantization Sensitivity. The Qwen-7B employs advanced pre-training techniques that reduce parameter redundancy. A documented tradeoff of this efficiency is a higher sensitivity to low-bit quantization (3-bit and below), where it exhibits more pronounced performance degradation compared to previous 7B generations.",
            "Context Window Consistency. While the 7B model supports a native context window of 32,768 tokens, its performance degrades significantly more than the Qwen-8B (which uses YaRN scaling to reach 128K+) when handling massive document sets. Users must tradeoff deep long-document comprehension for the 7B's lower memory footprint.",
            "Conciseness vs. Contextual Nuance. Experiments show that the older 7B design prioritizes cleaner, easier-to-read, and more concise outputs. The tradeoff is a loss of the \"faithful and nuanced\" insights and richer context provided by the newer 8B and larger architectures.",
            "Agentic Capability Limitations. The 7B model shows a documented gap in its ability to follow complex multi-step instructions or navigate large software repositories, requiring tighter chunking and more finely tuned prompts to be effective",
            "Hardware Efficiency vs. Throughput. Running the 7B model on older hardware (e.g., 8GB VRAM cards) is possible but results in a tradeoff of throughput. Modern inference techniques like continuous batching and PagedAttention are less effective at this scale than on the larger, more parallelizable MoE models.",
            "Decoding Strategy Rigidity. The 7B model is highly sensitive to sampling parameters; specifically, using greedy decoding (temperature=0) can lead to severe repetition loops and \"endless repetitions\". To maintain performance, users must tradeoff predictability for more complex sampling-based generation."
          ],
          "ethicalConsiderations": [
            {
              "name": "Algorithmic and Cultural Bias. As a model trained on 36 trillion tokens across 119 languages, Qwen-7B may still reflect societal biases, stereotypes, or representational harms present in its training data.",
              "mitigationStrategy": "Use the Qwen-Gender framework or Chain-of-Thought (CoT) prompting to detect and reduce implicit biases in generated text."
            },
            {
              "name": "Vulnerability to Adversarial Attacks (Jailbreaking). Despite safety tuning, the 7B model can be susceptible to \"Prompt Hacking\" or \"Jailbreaking\" where users bypass safety constraints to generate toxic or illegal content.",
```

```
              "mitigationStrategy": "Implement Qwen3Guard as an input/output filter to classify and
block unsafe queries or responses in real-time"
            },
            {
              "name": "Misinformation or Hallucinations. The model can fabricate false or misleading
information, especially regarding sensitive topics like government actions or historical events.",
              "mitigationStrategy": "Explicitly instruct the model to \"Prioritize Safety\" in the
prompt and use Retrieval-Augmented Generation (RAG) to ground responses in verified external documents."
            },
            {
              "name": "Privacy, Sensitive or Personally Identifiable Information (PII)Content Leakage.
If such data was present in the pre-training corpus, this risk for generation od such data is
possible.",
              "mitigationStrategy": "Deploy the model locally using tools like Ollama to ensure
sensitive data stays within a secure environment, and apply regex-based PII scrubbing to outputs."
            },
            {
              "name": "Environmental Impact (Inference Energy). Continuous large-scale deployment of
even mid-sized models like the 7B contributes to significant energy consumption and carbon footprints.",
              "mitigationStrategy": "Utilize 4-bit quantization and low-latency inference engines to
reduce the FLOPs required per token, minimizing the power draw per query."
            },
            {
              "name": "Instruction Misalignment. In-context learning can sometimes lead to \"emergent
misalignment\", where the model prioritizes following a user's conversational style over established
safety boundaries.",
              "mitigationStrategy": "Standardize output formats using system prompts and utilize the \"h
ard switch\" to disable the model's internal thinking mode when maximum safety and predictability are
required."
            }
          ],
          "fairnessAssessments": [
            {
              "groupAtRisk": "People identified by characteristics such as race, gender, and disability
status.",
              "harms": "The model was found to produce discriminatory outcomes across protected
characteristics, including race, gender, and disability status. For example, individuals categorized as
\"gypsy\" or \"mute\" were incorrectly labeled as untrustworthy in task assignment scenarios.",
              "mitigationStrategy": "Researchers recommend using Reinforcement Learning from Artificial
Intelligence Feedback (RLAIF) and rule-based rewards to align the model with specific legal standards
like the EU AI Act."
            },
            {
              "groupAtRisk": "Non-English/Non-Chinese speakers, speakers of regional dialects or
specific geographic regions (e.g., Southeast Asia or the Middle East) on thinking or \"reasoning\"
tasks.",
              "harms": "Quality-of-Service Harm: The model may provide high-quality, nuanced reasoning
in English or Mandarin but offer oversimplified, factually incorrect, or \"hallucinated\" information
when queried in other supported languages.",
              "mitigationStrategy":
"Cross-Lingual Alignment: Developers can use multilingual Supervised Fine-Tuning (SFT). By training on
additional high-quality, parallel corpora from other languages on \"reasoning capabilities\" (e.g.,
logic, math, coding)."
            }
          ],
          "environmentalConsiderations": {
            "energyConsumptions": [
              {
                "activity": "training",
                "energyProviders": [
                  {
                    "description": "Meta data-center, US-East",
                    "organization": {
                      "name": "Fake.ai",
```

```
                "address": {
                  "country": "United States",
                  "region": "New Jersey",
                  "locality": "Newark"
                }
              },
              "energySource": "natural-gas",
              "energyProvided": {
                "value": 0.4,
                "unit": "kWh"
              }
            }
          ],
          "activityEnergyCost": {
            "value": 0.4,
            "unit": "kWh"
          },
          "co2CostEquivalent": {
            "value": 31.22,
            "unit": "tCO2eq"
          },
          "co2CostOffset": {
            "value": 31.22,
            "unit": "tCO2eq"
          }
        }
      ]
    }
  },
  "properties": [
    {
      "name": "cdx:ai-ml:model:parameter:count",
      "value": "7B"
    },
    {
      "name": "cdx:ai-ml:model:parameter:tune_method",
      "value": "sft"
    },
    {
      "name": "cdx:ai-ml:model:parameter:tune_method",
      "value": "rlhf"
    },
    {
      "name": "cdx:ai-ml:model:hyperparameter:num_hidden_layers",
      "value": "32"
    },
    {
      "name": "cdx:ai-ml:model:hyperparameter:hidden_size",
      "value": "4096"
    },
    {
      "name": "cdx:ai-ml:model:hyperparameter:context_length",
      "value": "8192"
    },
    {
      "name": "cdx:ai-ml:model:hyperparameter:vocab_size",
      "value": "151936"
    },
    {
      "name": "cdx:ai-ml:model:hyperparameter:quantization",
      "value": "BF16"
    },
    {
      "name": "cdx:ai-ml:model:languages",
```

```
            "value": "en,fr,de,it,ja,zh"
          }
        ]
      }
    }
  },
  "components": [
    {
      "name": "Qwen3-8B-Q4_K_M.gguf",
      "type": "machine-learning-model",
      "bom-ref": "pkg:huggingface/Qwen/Qwen3-8B-GGUF@7c41481#Qwen3-8B-Q4_K_M.gguf",
      "purl": "pkg:huggingface/Qwen/Qwen3-8B-GGUF@7c41481f57cb95916b40956ab2f0b139b296d974#Qwen3-8B-
Q4_K_M.gguf",
      "version": "7c41481f57cb95916b40956ab2f0b139b296d974"
    },
    {
      "name": "UltraFeed-back dataset",
      "type": "data",
      "bom-ref": "pkg:huggingface/openbmb/UltraFeedback@40b4365",
      "purl": "pkg:huggingface/openbmb/UltraFeedback@40b436560ca83a8dba36114c22ab3c66e43f6d5e"
    },
    {
      "name": "UltraFeed-back dataset",
      "type": "data",
      "bom-ref": "pkg:huggingface/snorkelai/Snorkel-Mistral-PairRM-DPO@07af5d0a",
      "purl": "pkg:huggingface/snorkelai/Snorkel-Mistral-PairRM-
DPO@07af5d0a875b4c692dfaff6c675b10af07b45511"
    }
  ],
  "compositions": [
    {
      "aggregate": "complete",
      "assemblies": [
        "pkg:huggingface/Qwen/Qwen-7B@ef3c5c9"
      ]
    }
  ],
  "formulation": [
    {
      "components": [
        {
          "type": "container",
          "name": "h100-training-image",
          "version": "25.03-py3-igpu",
          "bom-ref": "pkg:oci/nvidia-pytorch@sha256:f398a0",
          "purl": "pkg:oci/nvidia-
pytorch@sha256:f398a0955ec5fcf9e3bbf77610225ff4e953e137423ab248e2bf32cd4971a1dc?repository_url=nvcr.io/
nvidia/pytorch&tag=25.03-py3-igpu"
        },
        {
          "type": "library",
          "name": "nvidia-cuda-runtime",
          "version": "12.2.0",
          "bom-ref": "pkg:generic/nvidia-cuda-runtime@12.2.0",
          "purl": "pkg:generic/nvidia-cuda-runtime@12.2.0"
        },
        {
          "type": "library",
          "name": "pytorch",
          "version": "2.10.0",
          "bom-ref": "pkg:pypi/pytorch@2.10.0",
          "purl": "pkg:pypi/pytorch@2.10.0"
        },
        {
```

```
        "type": "library",
        "name": "cuda-toolkit",
        "version": "13.1.1",
        "bom-ref": "pkg:pypi/cuda-toolkit@13.1.1",
        "purl": "pkg:pypi/cuda-toolkit@13.1.1"
      },
      {
        "type": "library",
        "name": "nccl",
        "version": "2.19.3",
        "bom-ref": "pkg:generic/nccl@2.29.2",
        "purl": "pkg:generic/nccl@2.29.2"
      },
      {
        "type": "device",
        "name": "NVIDIA H100 Tensor Core GPU",
        "description": "NVIDIA H100 Tensor Core GPU PCIe Device",
        "bom-ref": "nvidia-h100-pcie-gpu-1"
      }
    ],
    "workflows": [
      {
        "name": "Model training workflow",
        "bom-ref": "workflow-a3f9e2",
        "uid": "a3f9e2b1-7d8c-4a5f-9e6b-3c2d1a0b8f7e",
        "description": "Describes the tasks used for training the model described by the ML-BOM.",
        "taskTypes": [ "build" ],
        "runtimeTopology": [
          {
            "ref": "pkg:oci/nvidia-pytorch@sha256:f398a0",
            "dependsOn": [ "nvidia-h100-pcie-gpu-1" ],
            "provides": [
              "cuda-toolkit",
              "pkg:oci/nvidia-pytorch@sha256:f398a0",
              "pkg:pypi/pytorch@2.10.0",
              "pkg:generic/nccl@2.29.2"
            ]
          }
        ]
      }
    ]
  }
}
```